

Assignment 1: Market Risk (Volatility, Value-At-Risk and Expected Shortfall)

Total markdown character count across all three assignments: 29,672 / 30,000.

I hold three equally weighted US large caps from different sectors, Apple, Pfizer and Exxon, and I measure them against the S&P 500. Every VaR and ES number below is 1-day at 99%, and the Sharpe ratios assume a 4% risk-free rate.

```
In [1]: import os
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from scipy import stats
from arch import arch_model
import yfinance as yf
try:
    import seaborn as sns
except Exception:
    sns = None
from IPython.display import display
np.random.seed(42)

plt.rcParams.update({'figure.figsize': (13, 5), 'axes.grid': True, 'grid.alpha': 0.3,
                      'axes.spines.top': False, 'axes.spines.right': False, 'font.size': 12})
BLUE, RED, GREEN = '#2C7BB6', '#D7191C', '#1A9641'

TICKERS = ["AAPL", "PFE", "XOM"]
BENCH = "^GSPC"
WEIGHTS = np.array([1/3, 1/3, 1/3])
ALPHA = 0.01
Z = stats.norm.ppf(ALPHA)
RF_ANNUAL = 0.04

START = "2023-07-01"
END = "2026-07-01"
CACHE = "A1/my_portfolio.csv"
assert np.isclose(WEIGHTS.sum(), 1), "weights must sum to 1"
```

1 · Data, returns & cleaning

I load three years of daily adjusted closes from a fixed, dated CSV snapshot, so every rerun reproduces. Single names use log returns, the portfolio simple returns, since log returns do not add across assets and a weighted sum would leak an aggregation error into P&L.

```
In [2]: allsyms = TICKERS + [BENCH]
if os.path.exists(CACHE):
    px = pd.read_csv(CACHE, index_col=0, parse_dates=True)
    print(f"Loaded snapshot: {CACHE}")
else:
    px = yf.download(allsyms, start=START, end=END, auto_adjust=True)["Close"]
    px = px[allsyms]
    px.to_csv(CACHE)
    print(f"Downloaded {START} -> {END} from Yahoo Finance, saved to {CACHE}")
px = px[allsyms]

print("Rows:", len(px), "| Missing per column:\n", px.isna().sum().to_string())
px = px.ffill().dropna()

rets = np.log(px / px.shift(1)).dropna()

stocks = TICKERS
r_stk = rets[stocks]
r_mkt = rets[BENCH]

simple_stk = px[stocks].pct_change().dropna()
```

```

r_port = simple_stk @ WEIGHTS

med = rets.median()
mad = (rets - med).abs().median()
z_mad = 0.6745 * (rets - med) / mad
flagged = z_mad.abs() > 5
print("\n|z_MAD|>5 outliers per column (retained as genuine tail risk):\n",
      flagged.sum().to_string())
print("\nNamed outlier days (date | asset | move | robust z):")
for col in allsyms:
    for dt in rets.index[flagged[col]]:
        print(f" {dt.date()} {col:<6} {rets.loc[dt, col]*100:+6.2f}% z_MAD={z_mad.loc[dt, col]:+.1f}")

fig, ax = plt.subplots(1, 2, figsize=(12, 3.2))
(px[stocks] / px[stocks].iloc[0]).plot(ax=ax[0], title="Normalised prices (start=1)")
r_port.plot(ax=ax[1], title="Portfolio daily simple return", lw=.7)
plt.tight_layout(); plt.show()

```

Loaded snapshot: A1/my_portfolio.csv

Rows: 751 | Missing per column:

```

AAPL      0
PFE        0
XOM        0
^GSPC      0

```

|z_MAD|>5 outliers per column (retained as genuine tail risk):

```

AAPL      6
PFE        3
XOM        1
^GSPC      4

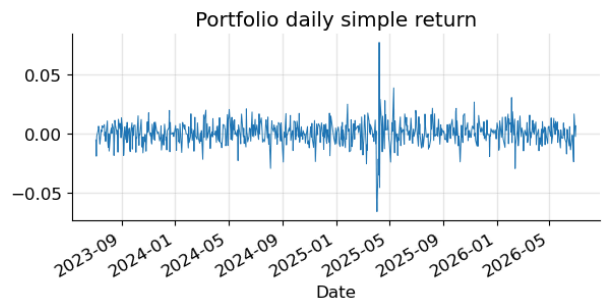
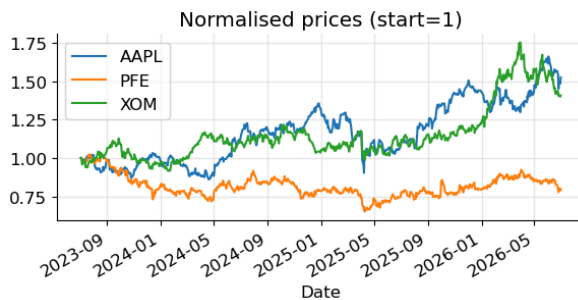
```

Named outlier days (date | asset | move | robust z):

```

2024-06-11 AAPL      +7.01% z_MAD=+6.0
2025-04-03 AAPL     -9.70% z_MAD=-8.5
2025-04-04 AAPL     -7.57% z_MAD=-6.7
2025-04-09 AAPL    +14.26% z_MAD=+12.3
2025-05-12 AAPL     +6.12% z_MAD=+5.2
2026-06-25 AAPL     -6.31% z_MAD=-5.6
2023-12-13 PFE      -6.95% z_MAD=-5.4
2025-09-30 PFE      +6.61% z_MAD=+5.1
2025-10-01 PFE      +6.57% z_MAD=+5.1
2025-04-04 XOM      -7.47% z_MAD=-5.8
2025-04-03 ^GSPC    -4.96% z_MAD=-7.3
2025-04-04 ^GSPC    -6.16% z_MAD=-9.0
2025-04-09 ^GSPC    +9.09% z_MAD=+12.9
2025-04-10 ^GSPC    -3.52% z_MAD=-5.2

```



Interpretation

The snapshot holds 751 trading days, from 2023-07-03 to 2026-06-30, with nothing missing, so the forward fill only guards against halts.

I screen outliers with a median/MAD z-score, not mean/std, because the standard deviation gets inflated by the very shocks I want to find and then hides them. It flags 14 days (AAPL 6, PFE 3, XOM 1, S&P 500 4), not scattered. Eight fall in one week of April 2025. AAPL dropped 9.70% on 3 April and 7.57% on 4 April, then jumped 14.26% on 9 April, while the S&P did -4.96%, -6.16% and +9.09%. The rest are company news, like PFE falling 6.95% on 13 December 2023.

These are real events, not data errors, so I keep all 14, the tail the VaR, ES and GARCH sections measure. Dropping them would flatter every number below.

2 · Annualised volatility & correlation

```
In [3]: summary = pd.DataFrame({
    'Ann. return': r_stk.mean() * 252,
    'Ann. vol':    r_stk.std() * np.sqrt(252),
})
summary['Sharpe (rf=4%)'] = (summary['Ann. return'] - RF_ANNUAL) / summary['Ann. vol']

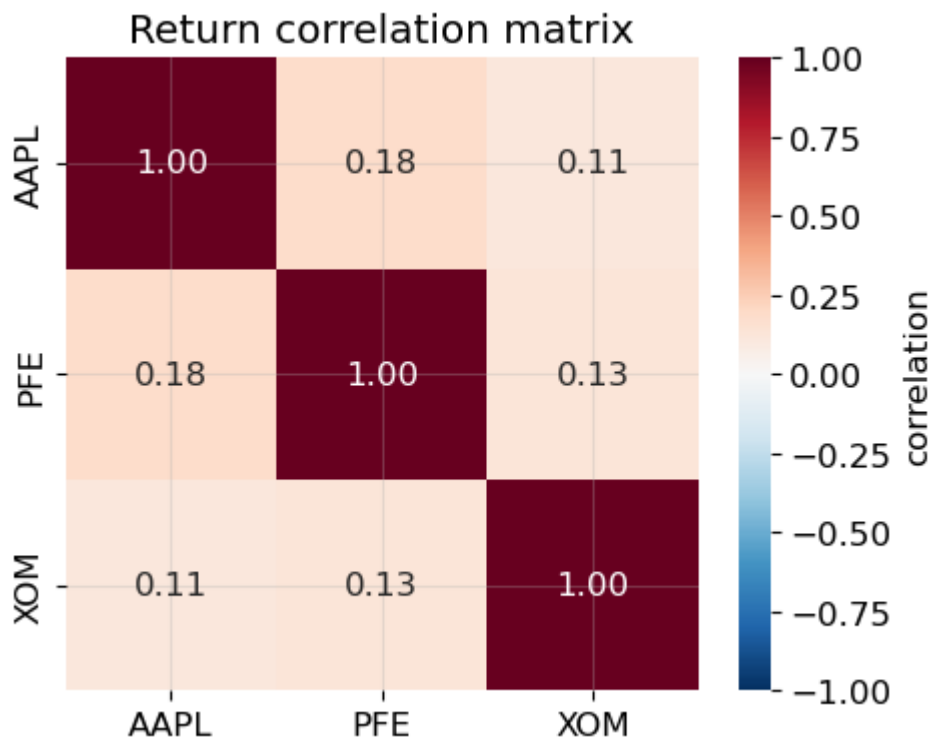
def max_drawdown(series):
    cum = (1 + series.pct_change().fillna(0)).cumprod()
    return (cum / cum.cummax() - 1).min()
summary['Max drawdown'] = px[stocks].apply(max_drawdown)
print('Per-stock risk summary:')
display(summary.sort_values('Ann. vol').round(3))

corr = r_stk.corr()
avg_corr = corr.values[np.triu_indices(len(stocks), 1)].mean()
fig, ax = plt.subplots(figsize=(5, 4))
if sns is not None:
    sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0, vmin=-1, vmax=1,
                square=True, cbar_kws={'label': 'correlation'}, ax=ax)
else:
    im = ax.imshow(corr, vmin=-1, vmax=1, cmap='RdBu_r'); fig.colorbar(im, fraction=.046)
    ax.set_xticks(range(len(stocks))); ax.set_xticklabels(stocks)
    ax.set_yticks(range(len(stocks))); ax.set_yticklabels(stocks)
ax.set_title('Return correlation matrix'); plt.tight_layout(); plt.show()
print(f'Average pairwise correlation: {avg_corr:.2f}')

ann_vol = r_stk.std() * np.sqrt(252)
port_vol = r_port.std() * np.sqrt(252)
div_ratio = (WEIGHTS @ ann_vol) / port_vol
print(f'Portfolio vol: {port_vol*100:.1f}% | Diversification ratio: {div_ratio:.2f}')
```

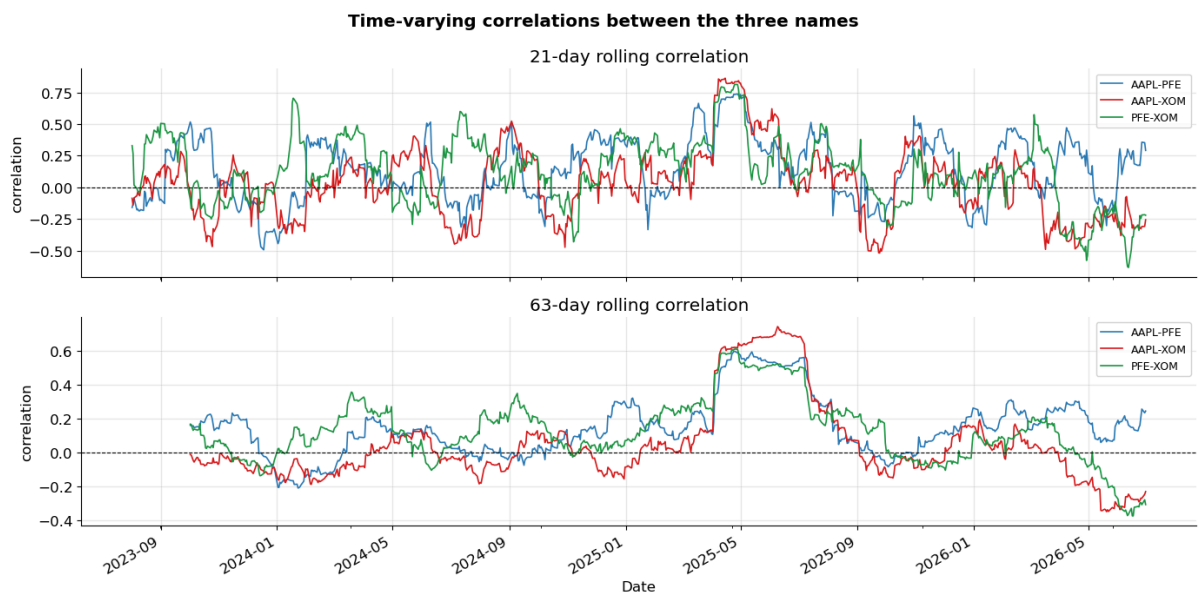
Per-stock risk summary:

	Ann. return	Ann. vol	Sharpe (rf=4%)	Max drawdown
XOM	0.115	0.233	0.321	-0.201
PFE	-0.078	0.242	-0.490	-0.364
AAPL	0.142	0.262	0.388	-0.334



Average pairwise correlation: 0.14
 Portfolio vol: 16.1% | Diversification ratio: 1.53

```
In [4]: from itertools import combinations
pairs = list(combinations(stocks, 2))
fig, axes = plt.subplots(2, 1, figsize=(14, 7), sharex=True)
for ax, window, title in zip(axes, [21, 63], ['21-day rolling', '63-day rolling']):
    for (a, b), col in zip(pairs, [BLUE, RED, GREEN]):
        rets[a].rolling(window).corr(rets[b]).plot(ax=ax, color=col, lw=1.2, label=f'{a}-{b}')
    ax.axhline(0, color='black', lw=.8, ls='--')
    ax.set_title(f'{title} correlation'); ax.set_ylabel('correlation'); ax.legend(fontsize=9)
plt.suptitle('Time-varying correlations between the three names', fontweight='bold')
plt.tight_layout(); plt.show()
```



Interpretation

The standalone volatilities sit close, AAPL 26.2%, PFE 24.2% and XOM 23.3% annualised, all normal for large caps. The portfolio lands at just 16.1%, a diversification ratio of 1.53, about a third less volatile than the weighted average of its parts. That comes from the names barely moving together, average pairwise correlation only 0.14, which fits three sectors on different drivers, rates and AI, drug pipelines, and crude.

Low correlation is not good performance, though. PFE returned -0.078 a year for a Sharpe of -0.49, a drag on return while still a diversifier. The rolling chart shows the catch, because those correlations wander and jump together in sell-offs, so 1.53 is a calm-market number that shrinks when I need it most.

3 · Beta, systematic vs. idiosyncratic risk, and stylised facts

I run the stylised-fact tests on the portfolio series rather than per name, since the portfolio is what the VaR sections model.

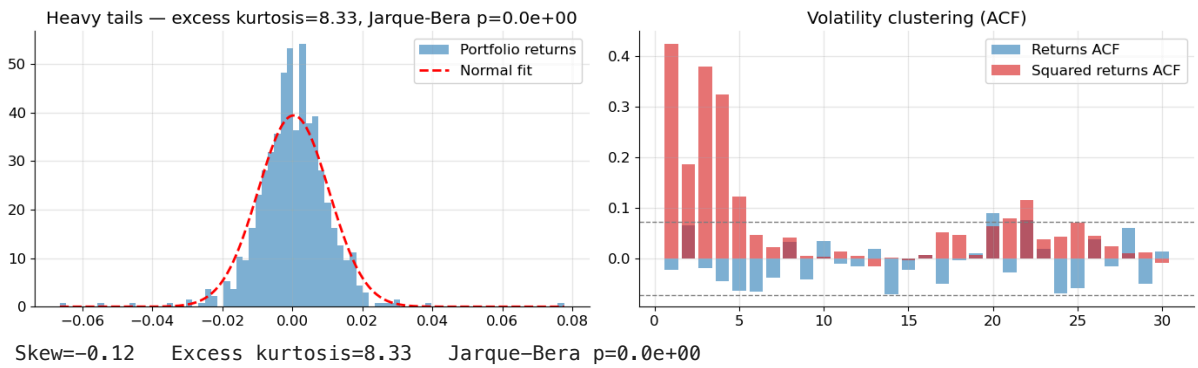
```
In [5]: rows = []
for s in stocks:
    slope, intercept, rval, pval, se = stats.linregress(r_mkt, r_stk[s])
    rows.append({'Stock': s,
                'Ann. vol %': r_stk[s].std() * np.sqrt(252) * 100,
                'Beta': slope,
                'Beta SE': se,
                'Beta p': pval,
                'Systematic R2': rval**2,
                'Idiosyncratic': 1 - rval**2,
                'Alpha ann %': intercept * 252 * 100,
                'Excess kurtosis': stats.kurtosis(r_stk[s], fisher=True)})
summary_df = pd.DataFrame(rows).set_index('Stock')
print(f'Per-stock beta vs {BENCH} (a broad index would sit near 1, which is the sanity check):')
display(summary_df.round(3))
```

Per-stock beta vs ^GSPC (a broad index would sit near 1, which is the sanity check):

	Ann. vol %	Beta	Beta SE	Beta p	Systematic R2	Idiosyncratic	Alpha ann %	Excess kurtosis
Stock								
AAPL	26.203	1.120	0.049	0.0	0.411	0.589	-5.430	10.215
PFE	24.161	0.391	0.057	0.0	0.059	0.941	-14.677	2.082
XOM	23.272	0.242	0.056	0.0	0.024	0.976	7.223	1.588

```
In [6]: r = r_port
jb_stat, jb_p = stats.jarque_bera(r)

fig = plt.figure(figsize=(14, 4))
ax1 = fig.add_subplot(1, 2, 1)
ax1.hist(r, bins=80, density=True, alpha=0.6, color=BLUE, label='Portfolio returns')
x = np.linspace(r.min(), r.max(), 400)
ax1.plot(x, stats.norm.pdf(x, r.mean(), r.std()), 'r--', lw=2, label='Normal fit')
ax1.set_title(f'Heavy tails - excess kurtosis={r.kurtosis():.2f}, Jarque-Bera p={jb_p:.1e}')
ax1.legend()
ax2 = fig.add_subplot(1, 2, 2)
lags = range(1, 31)
ax2.bar(lags, [r.autocorr(l) for l in lags], alpha=0.6, color=BLUE, label='Returns ACF')
ax2.bar(lags, [(r**2).autocorr(l) for l in lags], alpha=0.6, color=RED, label='Squared returns ACF')
ci = 1.96 / np.sqrt(len(r))
ax2.axhline(ci, color='gray', ls='--', lw=1); ax2.axhline(-ci, color='gray', ls='--', lw=1)
ax2.set_title('Volatility clustering (ACF)'); ax2.legend()
plt.tight_layout(); plt.show()
print(f'Skew={r.skew():.2f} Excess kurtosis={r.kurtosis():.2f} Jarque-Bera p={jb_p:.1e}')
```



Interpretation

The betas split the trio sharply. AAPL at 1.12 is 41% systematic and carries nearly all my market exposure, while PFE at 0.391 and XOM at 0.242 are 94% and 98% idiosyncratic, the diversifiers behind my 16.1% volatility.

XOM's 0.242 is the one I did not trust at first, below the 0.3 to 2 band I expect for a large cap, which usually flags a bad index or data, so I checked it. A standard error of 0.056 and a p-value near zero say the estimate is precise and not zero, and the S&P is the right index for a US book. The low beta is real, because XOM tracks crude and crude did not track the S&P over these three years.

The portfolio also fails the Normal, with excess kurtosis 8.33, AAPL alone at 10.215, and a Jarque-Bera p of zero to machine precision. The ACF plot separates the two effects, with nothing past lag 1 in raw returns but squared returns significant for many lags, so returns are unpredictable in direction yet predictable in size, which breaks a constant-Normal VaR.

4 · 1-day 99% VaR by three methods

The one design choice worth flagging is the Monte Carlo. A Normal draw would just re-run parametric's assumption, so instead I draw 200,000 scenarios from a multivariate Student-t ($\nu = 5$) scaled to the sample covariance, changing only the tail shape. A Normal MC runs alongside as a check.

```
In [7]: mu_p, sig_p = r_port.mean(), r_port.std()

var_param = -(mu_p + Z*sig_p)

var_hist = -np.percentile(r_port, 100*ALPHA)

rng = np.random.default_rng(42)
nu_mc = 5
mean = simple_stk.mean().values
cov = simple_stk.cov().values
Sig = cov * (nu_mc - 2) / nu_mc
L = np.linalg.cholesky(Sig)
N = 200_000
znorm = rng.standard_normal((N, len(stocks))) @ L.T
gchi = rng.chisquare(nu_mc, N) / nu_mc
sims = (mean + znorm / np.sqrt(gchi)[:, None]) @ WEIGHTS
var_mc = -np.percentile(sims, 100*ALPHA)

vardf = pd.DataFrame({"1-day 99% VaR": [var_param, var_hist, var_mc],
                      index=["Parametric (Normal)", "Historical", f"Monte Carlo (t, nu={nu_mc})"]})
print((vardf*100).round(2).astype(str).add(" %").to_string())
spread = (vardf.max().iloc[0]-vardf.min().iloc[0]) / vardf.mean().iloc[0]
```

```

print(f"\nMax-min spread as % of mean VaR: {spread*100:.0f}%")

sims_norm = rng.multivariate_normal(mean, cov, size=N) @ WEIGHTS
var_mc_norm = -np.percentile(sims_norm, 100*ALPHA)
print(f"(A Normal MC gives {var_mc_norm*100:.2f}%, essentially the parametric "
      f"{var_param*100:.2f}% -> it only repeats the parametric method, so the "
      f"fat-tailed t is what makes MC a real third estimate.)")

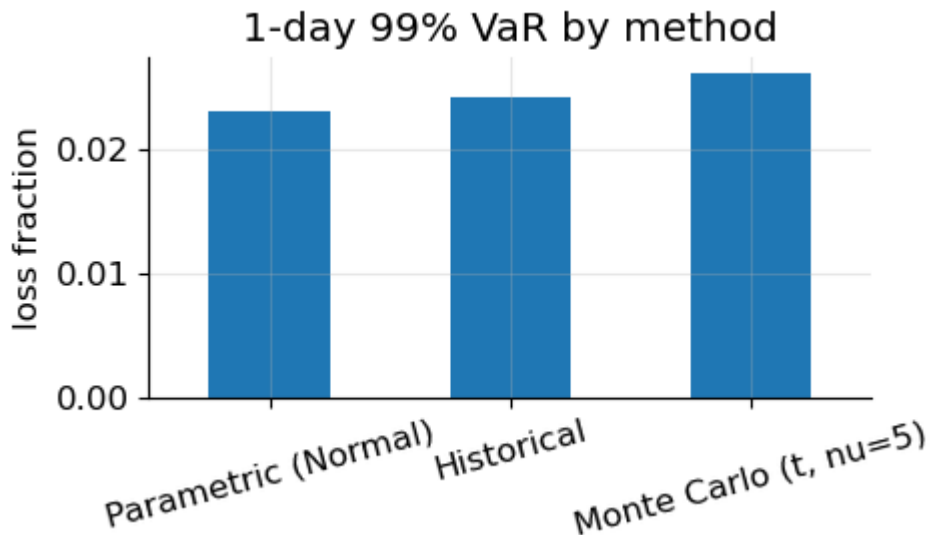
vardf.plot.bar(legend=False, figsize=(5,3), title="1-day 99% VaR by method", rot=15)
plt.ylabel("loss fraction"); plt.tight_layout(); plt.show()

```

	1-day 99% VaR
Parametric (Normal)	2.32 %
Historical	2.42 %
Monte Carlo (t, nu=5)	2.62 %

Max-min spread as % of mean VaR: 12%

(A Normal MC gives 2.32%, essentially the parametric 2.32% -> it only repeats the parametric method, so the fat-tailed t is what makes MC a real third estimate.)



Interpretation

The three methods disagree sensibly. Parametric gives 2.32%, historical 2.42% and the Student-t Monte Carlo 2.62%, a spread of about 12% of the mean VaR, all low single digits.

The ordering is the informative part. The t-MC is highest because its fat tails weight the extreme-loss region more than a Normal allows, historical sits in the middle because it feels the actual April 2025 days, and parametric is lowest because the Normal tail is thinnest, which lines up with the kurtosis of 8.33. The Normal MC confirms it, matching parametric at 2.32%, so it would have been a copy.

So my VaR is really a range from 2.32% to 2.62%, and which end I quote is a modelling choice, not a measurement. My window is calm, though, so at a more extreme quantile or with a real crash the fat-tailed estimate would pull much further away and 12% would look small.

5 · Expected Shortfall & sub-additivity

For the super-additivity test I compare each name's VaR at its portfolio weight against the portfolio VaR, since full positions against a one-third weighting would rig it. I run five confidence levels, since one can pass by luck.

```
In [8]: tail = r_port[r_port <= -var_hist]
es_hist = -tail.mean()
es_param = -(mu_p - sig_p*stats.norm.pdf(Z)/ALPHA)
print(f"Historical VaR {var_hist*100:.2f}% ES {es_hist*100:.2f}% ES/VaR {es_hist/var_hist:.2f}")
print(f"Parametric VaR {var_param*100:.2f}% ES {es_param*100:.2f}% ES/VaR {es_param/var_param:.2f}")

print("\nSuper-additivity across confidence levels (historical, weight-consistent):")
print(f" {'conf':>6} {'sum comp':>9} {'portfolio':>10} {'benefit':>9} verdict")
for conf in [0.90, 0.95, 0.99, 0.995, 0.999]:
    a = 1 - conf
    comp = sum(-np.percentile(WEIGHTS[i]*simple_stk[s], 100*a) for i, s in enumerate(stocks))
    port = -np.percentile(r_port, 100*a)
    verdict = "sub-additive" if port <= comp else "SUPER-ADDITIVE (VaR fails!)"
    print(f" {conf*100:5.1f}% {comp*100:8.2f}% {port*100:9.2f}% {(comp-port)*100:7.2f}pp {verdict}")
```

```
Historical VaR 2.42% ES 3.89% ES/VaR 1.60
Parametric VaR 2.32% ES 2.66% ES/VaR 1.15
```

```
Super-additivity across confidence levels (historical, weight-consistent):
```

conf	sum comp	portfolio	benefit	verdict
90.0%	1.73%	1.06%	0.66pp	sub-additive
95.0%	2.42%	1.45%	0.97pp	sub-additive
99.0%	4.08%	2.42%	1.66pp	sub-additive
99.5%	4.70%	3.11%	1.59pp	sub-additive
99.9%	6.50%	5.70%	0.80pp	sub-additive

Interpretation

Historical ES is 3.89% against a VaR of 2.42%, so on the worst 1% of days my average loss is 1.6 times the VaR threshold.

Parametric ES is 2.66%, a ratio of 1.15 against the historical 1.60, yet the two VaRs are nearly identical at 2.32% and 2.42%. So the Normal assumption looks almost harmless at the threshold and badly wrong just past it, which is the real argument for ES.

On sub-additivity, my portfolio VaR sits below the summed component VaRs at every level from 90% to 99.9%, and the benefit grows from 0.66pp to a peak of 1.66pp at 99%, so diversification is rewarded and never turns super-additive here. But this is a fact about my sample, not a property of VaR, following from the 0.14 correlation, and with skewed or fat-tailed positions VaR can punish diversification where ES is coherent and cannot. I would not lean on the 99.9% row, where the benefit falls to 0.80pp on a tail of roughly 7 days out of 750, sampling noise.

6 · Time-varying volatility with GARCH(1,1)

Two choices to flag. I pick the stock with a Ljung-Box test for clustering across all three, taking the strongest rejection not the most volatile name, and I fit Student-t innovations checked with AIC. The VaR and ES use the fitted t quantile.

```
In [9]: from statsmodels.stats.diagnostic import acorr_ljungbox
print("Volatility-clustering test (Ljung-Box on squared returns, lag 10):")
lb = {}
for s in stocks:
    lb[s] = acorr_ljungbox(r_stk[s]**2, lags=[10], return_df=True)["lb_pvalue"].iloc[0]
    print(f" {s:<6} LB(squared) p = {lb[s]:.3g}")
target = min(lb, key=lb.get)
print(f"-> GARCH target chosen by clustering evidence: {target}")

r_pct = r_stk[target] * 100
res = arch_model(r_pct, vol='Garch', p=1, q=1, dist='t', mean='Zero').fit(dis='off')
print(res.summary().tables[1])
```

```

w, a, b = res.params['omega'], res.params['alpha[1]'], res.params['beta[1]']
nu      = res.params['nu']
pv      = res.pvalues
persist = a + b
sig = lambda p: 'significant' if p < 0.05 else 'NOT significant'
print(f"\nParameter significance: omega p={pv['omega']:.3g} ({sig(pv['omega'])}), "
      f"alpha p={pv['alpha[1]']:.3g} ({sig(pv['alpha[1]'])}), "
      f"beta p={pv['beta[1]']:.3g} ({sig(pv['beta[1]'])}), nu p={pv['nu']:.3g}")

if persist < 1:
    uncond_vol = np.sqrt(w/(1-persist)) / 100 * np.sqrt(252)
    half_life  = np.log(0.5)/np.log(persist)
    print(f"Persistence (alpha+beta) = {persist:.3f} < 1 | long-run vol "
          f"{uncond_vol*100:.1f}%/yr | shock half-life {half_life:.1f} days | "
          f"Student-t dof nu = {nu:.2f}")
else:
    uncond_vol = half_life = np.nan
    print(f"Persistence (alpha+beta) = {persist:.3f} >= 1: near-integrated (IGARCH) "
          f"fit -> long-run variance UNDEFINED (boundary fit). Do NOT report an "
          f"unconditional vol or half-life here; treat the persistence itself as "
          f"the finding and investigate.")

res_norm = arch_model(r_pct, vol='Garch', p=1, q=1, dist='normal', mean='Zero').fit(displ='off')
print(f"AIC: Student-t {res.aic:.1f} vs Normal {res_norm.aic:.1f} -> "
      f"{'t preferred (fat tails earn their keep)' if res.aic < res_norm.aic else 'Normal preferred'}")

cond_vol = res.conditional_volatility
sig_fc   = np.sqrt(res.forecast(horizon=1).variance.iloc[-1, 0]) / 100
sig_bar  = r_stk[target].std()

q_raw = stats.t.ppf(ALPHA, nu)
scale = np.sqrt((nu - 2) / nu)
q_t   = q_raw * scale
es_t  = (stats.t.pdf(q_raw, nu)/ALPHA) * (nu + q_raw**2)/(nu - 1) * scale
print(f"Standardised-t 1% quantile = {q_t:.3f} (Normal z = {Z:.3f}) | "
      f"ES multiplier = {es_t:.3f} (Normal = {stats.norm.pdf(Z)/ALPHA:.3f}")

var_garch = -sig_fc * q_t
es_garch  = sig_fc * es_t
var_const = -sig_bar * q_t
es_const  = sig_bar * es_t
print(f"\n1-day GARCH cond. vol forecast: {sig_fc*100:.2f}% (constant {sig_bar*100:.2f}%)")
print(f"GARCH-t {target}: VaR {var_garch*100:.2f}% ES {es_garch*100:.2f}%")
print(f"Constant-t {target}: VaR {var_const*100:.2f}% ES {es_const*100:.2f}%")

fig, ax = plt.subplots(2, 1, figsize=(13, 6), sharex=True)
ax[0].plot(r_pct.index, r_pct, color=BLUE, lw=.5, alpha=.8)
ax[0].set_title(f'{target} returns (%)')
ax[1].fill_between(cond_vol.index, cond_vol, alpha=.4, color=RED)
ax[1].plot(cond_vol.index, cond_vol, color=RED, lw=1)
ax[1].set_title(f'{target} GARCH(1,1) conditional volatility (daily %)')
plt.tight_layout(); plt.show()

```

Volatility-clustering test (Ljung-Box on squared returns, lag 10):

AAPL LB(squared) p = 8.75e-27

PFE LB(squared) p = 0.00128

XOM LB(squared) p = 1.01e-15

-> GARCH target chosen by clustering evidence: AAPL

Volatility Model

	coef	std err	t	P> t	95.0% Conf. Int.
omega	0.4927	0.703	0.701	0.483	[-0.885, 1.870]
alpha[1]	0.1240	0.112	1.105	0.269	[-9.600e-02, 0.344]
beta[1]	0.7096	0.352	2.019	4.353e-02	[2.059e-02, 1.399]

Parameter significance: omega p=0.483 (NOT significant), alpha p=0.269 (NOT significant), beta p=0.0435 (significant), nu p=1.43e-13

Persistence (alpha+beta) = 0.834 < 1 | long-run vol 27.3%/yr | shock half-life 3.8 days | Student-t dof nu = 3.58

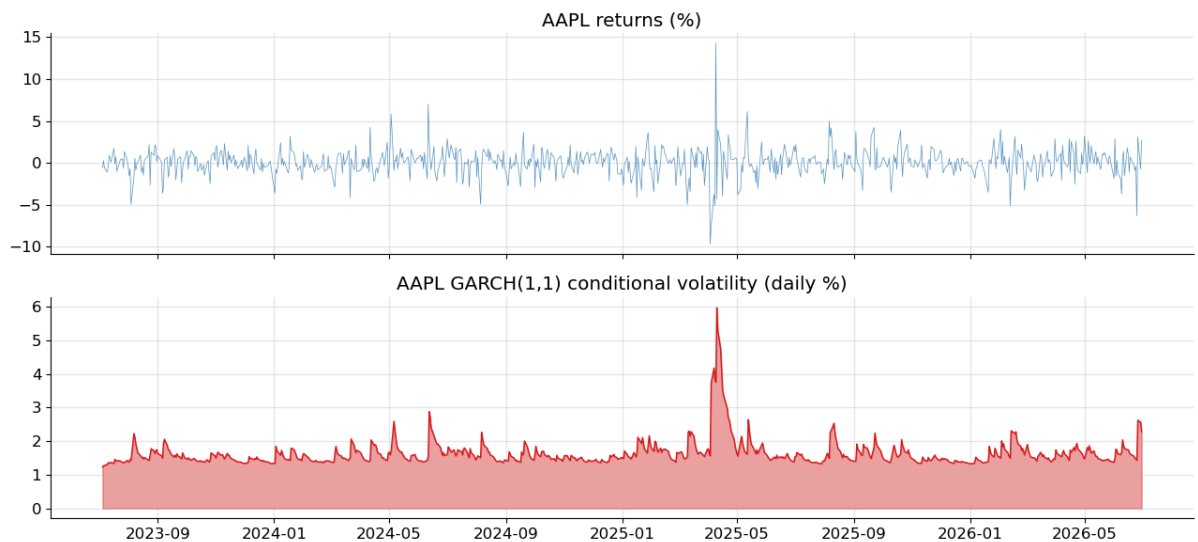
AIC: Student-t 2675.2 vs Normal 2784.5 -> t preferred (fat tails earn their keep)

Standardised-t 1% quantile = -2.659 (Normal z = -2.326) | ES multiplier = 3.830 (Normal = 2.665)

1-day GARCH cond. vol forecast: 2.25% (constant 1.65%)

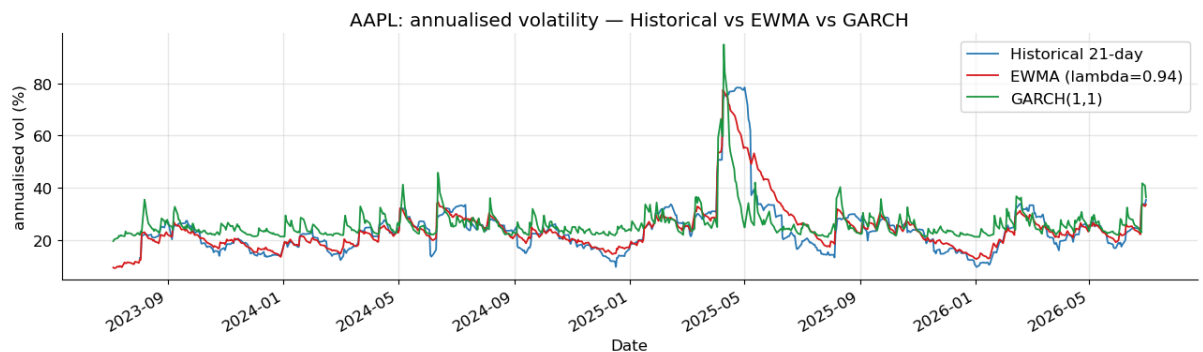
GARCH-t AAPL: VaR 5.99% ES 8.63%

Constant-t AAPL: VaR 4.39% ES 6.32%



```
In [10]: lam = 0.94
r_t = r_stk[target]
hist_vol = r_t.rolling(21).std() * np.sqrt(252) * 100
ewma_var = r_t.copy(); ewma_var.iloc[0] = r_t.iloc[0]**2
for t in range(1, len(r_t)):
    ewma_var.iloc[t] = lam * ewma_var.iloc[t-1] + (1-lam) * r_t.iloc[t]**2
ewma_vol = np.sqrt(ewma_var) * np.sqrt(252) * 100
garch_vol = cond_vol * np.sqrt(252)

fig, ax = plt.subplots(figsize=(13, 4))
hist_vol.plot(ax=ax, color=BLUE, lw=1.3, label='Historical 21-day')
ewma_vol.plot(ax=ax, color=RED, lw=1.3, label=f'EWMA (lambda={lam})')
garch_vol.plot(ax=ax, color=GREEN, lw=1.3, label='GARCH(1,1)')
ax.set_title(f'{target}: annualised volatility — Historical vs EWMA vs GARCH')
ax.set_ylabel('annualised vol (%)'); ax.legend(); plt.tight_layout(); plt.show()
```



Interpretation

The Ljung-Box test picks the stock. AAPL rejects no-clustering hardest at $p = 8.75e-27$, against $1.01e-15$ for XOM and only 0.00128 for PFE, so PFE is barely clustered at almost the same volatility, and clustering is not volatility.

Persistence is 0.834 , from α 0.124 and β 0.7096 , below 1 , so the long-run variance is defined. It comes out at 27.3% a year, right next to AAPL's 26.2% sample volatility. The half-life is 3.8 days, lower than the 0.95 to 0.99 usual in long daily samples, probably because my window is short and April 2025 spiked and reversed fast.

In the coefficient table, β at $p = 0.0435$ and ν at $p = 1.43e-13$ are significant, but ω at $p = 0.483$ and α at $p = 0.269$ are not, with intervals crossing zero, so I lean on the joint persistence the VaR depends on. AIC backs the t at 2675.2 against 2784.5 , and $\nu = 3.58$ is very heavy.

The forecast is 2.25% daily against a 1.65% constant, both on the same t quantile so the gap isolates the regime. VaR is 5.99% against 4.39%, ES 8.63% against 6.32%, and a desk on the constant number is under-capitalised on AAPL by about a third today.

7 · Backtest & Basel traffic light

I roll a 500-day estimation window and count exceptions over a 250-day test block, the horizon Basel's zones use (Green 0 to 4, Yellow 5 to 9, Red 10 or more). The by-year check uses a 250-day window instead, since 500 days on my 750-day sample just reproduces the headline 250 days, so the two counts are not strictly comparable.

```
In [11]: from scipy.stats import chi2
win, test_n = 500, 250
series = r_port.values
exc = []
for t in range(len(series)-test_n, len(series)):
    hist = series[t-win:t]
    v = -(hist.mean() + Z*hist.std())
    exc.append(series[t] < -v)
exc = np.array(exc)
n_exc, expected = exc.sum(), ALPHA*test_n
zone = "GREEN" if n_exc <= 4 else "YELLOW" if n_exc <= 9 else "RED"
print(f"Basel zone read on 250 obs -> Exceptions: {n_exc} / {test_n}    "
      f"Expected: {expected:.1f}    Zone: {zone}")

diag_win = 250
fi, fe = [], []
for t in range(diag_win, len(series)):
    hist = series[t-diag_win:t]
    v = -(hist.mean() + Z*hist.std())
    fi.append(r_port.index[t]); fe.append(series[t] < -v)
full = pd.Series(fe, index=pd.DatetimeIndex(fi))
by_year = full.groupby(full.index.year).agg(exceptions='sum', days='count')
by_year['rate %'] = (by_year['exceptions']/by_year['days']*100).round(2)
print(f"\nBreaches by year, {diag_win}-day estimation window ({len(full)} days "
      f"covered vs {len(series)-win} on a {win}-day window):")
print(by_year.to_string())
worst = int(by_year['exceptions'].max())
worst_yr = int(by_year['exceptions'].idxmax())
worst_days = int(by_year.loc[worst_yr, 'days'])
print(f"Worst single year: {worst} breaches in {worst_yr} ({worst_days} days covered).")

def kupiec_pof(x, n, alpha):
    p, p_hat = alpha, x/n
    if p_hat in (0, 1): return np.nan, np.nan
    lr = -2*(x*np.log(p/p_hat) + (n-x)*np.log((1-p)/(1-p_hat)))
    return lr, chi2.sf(lr, df=1)

def christoffersen_ind(v):
    v = v.astype(int)
    n00=int(((v[:-1]==0)&(v[1:]==0)).sum()); n01=int(((v[:-1]==0)&(v[1:]==1)).sum())
    n10=int(((v[:-1]==1)&(v[1:]==0)).sum()); n11=int(((v[:-1]==1)&(v[1:]==1)).sum())
    pi01 = n01/(n00+n01) if (n00+n01) else 0.0
    pi11 = n11/(n10+n11) if (n10+n11) else 0.0
    pi = (n01+n11)/(n00+n01+n10+n11)
    if pi in (0,1) or pi01 in (0,1): return np.nan, np.nan, (pi01, pi11)
    sl = lambda x: np.log(max(x, 1e-300))
    ll0 = (n00+n10)*sl(1-pi) + (n01+n11)*sl(pi)
    ll1 = n00*sl(1-pi01)+n01*sl(pi01)+n10*sl(1-pi11)+n11*sl(pi11)
    lr = -2*(ll0-ll1)
    return lr, chi2.sf(lr, df=1), (pi01, pi11)

LR, p_uc = kupiec_pof(int(n_exc), test_n, ALPHA)
LR_ind, p_ind, _ = christoffersen_ind(exc)
print(f"\nKupiec POF          LR = {LR:.2f}    p = {p_uc:.2f}    (>3.84 rejects correct 99% coverage)")
if not np.isnan(LR_ind):
    LR_cc = LR + LR_ind
    print(f"Christoffersen ind. LR = {LR_ind:.2f}    p = {p_ind:.2f}    (>3.84 => exceptions cluster)")
    print(f"Conditional coverage LR_cc = {LR_cc:.2f}    (df=2; >5.99 => reject jointly)")
else:
    print("Christoffersen ind.: too few exceptions -> clustering not detectable ")
```

```

"(low power: ~2.5 breaches expected per 250 days at 99%).")
test_ret = r_port.iloc[-test_n:]
plt.figure(figsize=(11,2.8))
plt.plot(test_ret.index, test_ret.values, lw=.6, label="return")
plt.scatter(test_ret.index[exc], test_ret.values[exc], c="red", s=18, label="exception")
plt.title(f"Backtest - {n_exc} exceptions ({zone})"); plt.legend(); plt.tight_layout();
plt.show()

```

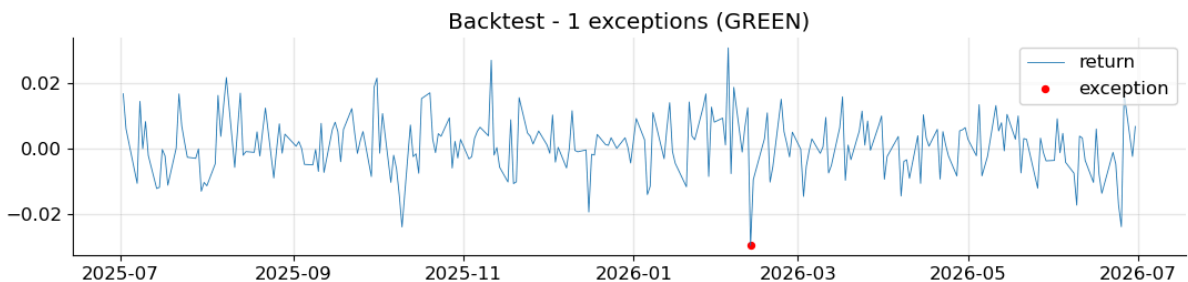
Basel zone read on 250 obs -> Exceptions: 1 / 250 Expected: 2.5 Zone: GREEN

Breaches by year, 250-day estimation window (500 days covered vs 250 on a 500-day window):

	exceptions	days	rate %
2024	3	127	2.36
2025	5	250	2.00
2026	2	123	1.63

Worst single year: 5 breaches in 2025 (250 days covered).

Kupiec POF LR = 1.18 p = 0.28 (>3.84 rejects correct 99% coverage)
 Christoffersen ind. LR = 0.01 p = 0.93 (>3.84 => exceptions cluster)
 Conditional coverage LR_cc = 1.18 (df=2; >5.99 => reject jointly)



Interpretation

The headline is clean and boring. One exception in 250 days against 2.5 expected is Basel GREEN. Kupiec gives LR = 1.18 with p = 0.28, so I cannot reject correct coverage, and Christoffersen's LR = 0.01 with p = 0.93 says the breach is isolated, not the start of a cluster.

The by-year table stops me believing it, the most interesting output here. Rolling the same VaR back gives 3 breaches in 2024, 5 in 2025 and 2 in 2026, and five on a 250-day year is Yellow. So my calm recent window is hiding a worse year, the April 2025 tariff week doing most of the work, and the GREEN really describes the last twelve months.

I should be careful, though. The by-year model uses the shorter window and every year runs above the 1% target, at 2.36%, 2.00% and 1.63%, so some of that is the noisier window misfiring. 2025 is not proof the 500-day model would have gone Yellow, but it shows my GREEN is fragile, changing zone the moment I change one modelling choice. Power is the deeper limit, since at 99% only about 2.5 breaches are expected a year, so one quiet block is weak evidence and my sample holds nothing like 2008 or March 2020. I read this GREEN as weak evidence, not validation.

8 · Assumptions, limitations & real-world implications

Every method leans on an assumption my data breaks. Parametric VaR and ES want Normal, independent returns I do not have, historical simulation assumes three years represent the future, my Monte Carlo only trades that for my own choice of nu = 5, and

beta assumes a stable link my drifting correlations would break in a crisis. Running all three makes that model risk visible instead of hiding it in one number.

The rest is about my sample, which covers one calm regime, so my clean backtest and diversification benefit are both measured where risk is easiest. A three-name book also leaves so much idiosyncratic risk that a market-risk framework only sees part of the danger. AAPL sets the buffer, while PFE and XOM need position limits for risk hedges cannot touch, a failed trial or a crude move. Risk management is not one number, and the discipline is knowing what each tool cannot see.

Assignment 2: Credit, Derivative, Liquidity & Operational Risk

Same three companies as before, AAPL, PFE and XOM. The data is a dated snapshot and every simulation is seeded, so this reproduces.

```
In [12]: import os, warnings, itertools
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import norm, skew, kurtosis
from scipy.optimize import fsolve
from IPython.display import display
warnings.filterwarnings('ignore')
np.random.seed(42)

plt.rcParams.update({'figure.figsize': (13, 5), 'axes.grid': True, 'grid.alpha': 0.3,
                      'axes.spines.top': False, 'axes.spines.right': False, 'font.size': 12})
BLUE, RED, GREEN, GOLD = '#2C7BB6', '#D7191C', '#1A9641', '#F39C12'

STOCKS = ['AAPL', 'PFE', 'XOM']
RF, LGD, RECOV = 0.04, 0.45, 0.55

px = pd.read_csv('A1/my_portfolio.csv', index_col=0, parse_dates=True)
rets = np.log(px / px.shift(1)).dropna()
SIGMA_E = (rets[STOCKS].std() * np.sqrt(252)).to_dict()

W = np.array([1/3, 1/3, 1/3])
r_port = px[STOCKS].pct_change().dropna() @ W
var_base = -(r_port.mean() + stats.norm.ppf(0.01) * r_port.std())

E = {'AAPL': 4.660445e12, 'PFE': 1.395224e11, 'XOM': 5.989863e11}
D = {'AAPL': 8.4711e10, 'PFE': 6.4731e10, 'XOM': 4.7661e10}
PRICE = {'AAPL': 317.31, 'PFE': 24.48, 'XOM': 144.51}
RATING = {'AAPL': 'AA+', 'PFE': 'A', 'XOM': 'AA-'}
SPREAD = {'AAPL': 0.0035, 'PFE': 0.0070, 'XOM': 0.0045}
RATING_PD = {'AA+': 0.0002, 'AA-': 0.0002, 'A': 0.0005}

print('Equity vol:', {k: round(v, 3) for k, v in SIGMA_E.items()})
print(f'Portfolio 1-day 99% VaR: {var_base*100:.2f}%')
print(f'\nPrice sample ends {px.index[-1].date()}. Hand-entered Merton inputs, '
      f'against that close:')
for s in STOCKS:
    print(f' {s:5s} market cap ${E[s]/1e12:.2f}tn total debt ${D[s]/1e9:.1f}bn '
          f'spot ${PRICE[s]:.2f} last close in sample ${px[s].iloc[-1]:.2f}')
```

Equity vol: {'AAPL': 0.262, 'PFE': 0.242, 'XOM': 0.233}

Portfolio 1-day 99% VaR: 2.32%

Price sample ends 2026-06-30. Hand-entered Merton inputs, against that close:

AAPL	market cap \$4.66tn	total debt \$84.7bn	spot \$317.31	last close in sample \$289.36
PFE	market cap \$0.14tn	total debt \$64.7bn	spot \$24.48	last close in sample \$24.08
XOM	market cap \$0.60tn	total debt \$47.7bn	spot \$144.51	last close in sample \$136.72

1 · Merton structural model & default probability

Equity is a call on the firm's assets struck at the debt face. I cannot see the assets, so I solve the two Merton equations for asset value and volatility from E, my equity volatility and D, giving the distance-to-default and $PD = N(-DD)$. E and D are each firm's market cap and total debt from Yahoo, pulled just after the close ending my return series, so AAPL's \$4.66tn cap sits on a \$317.31 spot not the \$289.36 my sample ends on, deliberate since Merton wants today's equity but the volatility spans three years. Ratings are S&P grades and the spreads indicative, not quoted CDS.

Three approximations bend the PD. The risk-free rate stands in for the asset drift, so my PD is risk-neutral and above the real-world frequency, and total debt as the barrier is conservative. But the debt is one zero-coupon bond maturing in a year, so default can only strike then.

```
In [13]: def merton_calibrate(E_obs, sigma_E, D_face, r=RF, T=1.0):
def eqs(x):
    V, sV = x
    d1 = (np.log(V/D_face) + (r + 0.5*sV**2)*T) / (sV*np.sqrt(T))
    d2 = d1 - sV*np.sqrt(T)
    return [V*norm.cdf(d1) - D_face*np.exp(-r*T)*norm.cdf(d2) - E_obs,
            norm.cdf(d1)*sV*V - sigma_E*E_obs]
    V, sV = fsolve(eqs, [E_obs + D_face, sigma_E*E_obs/(E_obs+D_face)])
    dd = (np.log(V/D_face) + (r - 0.5*sV**2)*T) / (sV*np.sqrt(T))
    return V, sV, dd, norm.cdf(-dd)

rows = []
for s in STOCKS:
    V, sV, dd, pd_ = merton_calibrate(E[s], SIGMA_E[s], D[s])
    cds_pd, rat_pd = SPREAD[s]/(1-RECOV), RATING_PD[RATING[s]]
    rows.append({'Firm': s, 'Rating': RATING[s], 'Spread (bp)': SPREAD[s]*1e4,
                'Leverage %': D[s]/(E[s]+D[s])*100,
                'Asset vol': sV, 'DD (Merton)': dd, 'PD Merton': pd_,
                'PD rating': rat_pd, 'PD CDS': cds_pd,
                'DD implied by CDS': -norm.ppf(cds_pd)})
merton_df = pd.DataFrame(rows).set_index('Firm')

show = merton_df.round({'Spread (bp)': 0, 'Leverage %': 1, 'Asset vol': 3,
                        'DD (Merton)': 2, 'DD implied by CDS': 2})
show['PD Merton'] = merton_df['PD Merton'].map('{:.2e}'.format)
show['PD rating'] = merton_df['PD rating'].map('{:.3%}'.format)
show['PD CDS'] = merton_df['PD CDS'].map('{:.3%}'.format)
display(show)

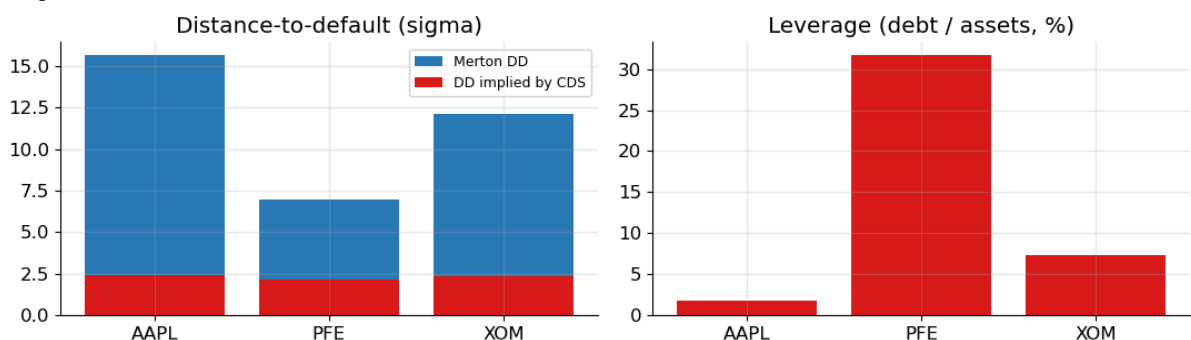
print('The same firm, three PD measures (1-year):')
for s in STOCKS:
    r_ = merton_df.loc[s]
    print(f" {s:5s} Merton {r_['PD Merton']:.2e} rating {r_['PD rating']:.3%} "
          f"CDS {r_['PD CDS']:.3%} | DD: model {r_['DD (Merton)']:.1f} sigma vs "
          f"CDS-implied {r_['DD implied by CDS']:.1f} sigma")

fig, ax = plt.subplots(1, 2, figsize=(11, 3.3))
ax[0].bar(STOCKS, [merton_df.loc[s, 'DD (Merton)'] for s in STOCKS], color=BLUE, label='Merton DD')
ax[0].bar(STOCKS, [merton_df.loc[s, 'DD implied by CDS'] for s in STOCKS], color=RED,
          label='DD implied by CDS')
ax[0].set_title('Distance-to-default (sigma)'); ax[0].legend(fontsize=9)
ax[1].bar(STOCKS, [D[s]/(E[s]+D[s])*100 for s in STOCKS], color=RED)
ax[1].set_title('Leverage (debt / assets, %)')
plt.tight_layout(); plt.show()
```

Firm	Rating	Spread (bp)	Leverage %	Asset vol	DD (Merton)	PD Merton	PD rating	PD CDS	DD implied by CDS
AAPL	AA+	35.0	1.8	0.258	15.66	1.52e-55	0.020%	0.778%	2.42
PFE	A	70.0	31.7	0.167	6.96	1.74e-12	0.050%	1.556%	2.16
XOM	AA-	45.0	7.4	0.216	12.13	3.88e-34	0.020%	1.000%	2.33

The same firm, three PD measures (1-year):

AAPL Merton 1.52e-55 rating 0.020% CDS 0.778% | DD: model 15.7 sigma vs CDS-implied 2.4 sigma
PFE Merton 1.74e-12 rating 0.050% CDS 1.556% | DD: model 7.0 sigma vs CDS-implied 2.2 sigma
XOM Merton 3.88e-34 rating 0.020% CDS 1.000% | DD: model 12.1 sigma vs CDS-implied 2.3 sigma



Interpretation

My structural PDs are microscopic. PFE is highest at about 1.7e-12 because it carries the most leverage, debt near 32% of assets, while AAPL sits 15.7 sigma out at around 1e-55.

For AAPL the market charges a 35bp spread, implying a 0.78% one-year PD, the rating tables put an AA+ default at about 0.02%, and Merton says 1e-55. So the market prices AAPL as if 2.4 sigma from default while Merton puts it 15.7 sigma out. The CDS number is too high because dividing the whole spread by one minus recovery blames it all on default, when much of a spread is risk premium and illiquidity.

Both my risk-neutral PDs should top the table's real-world 0.02%, and the CDS PD does at 0.78%, but Merton's 1e-55 sits far below, so the thin Normal tail swamps a bias that should have pushed it up. Real firms gap into default while this model only lets them drift.

2 · Loan book & Monte-Carlo portfolio loss

I run on the CDS-implied PDs. The structural ones are too small to lose anything worth simulating.

```
In [14]: book = pd.DataFrame(index=STOCKS)
book['EAD weight'] = np.array([D[s] for s in STOCKS]); book['EAD weight'] /= book['EAD weight'].sum()
```

```

book['PD (CDS-implied)'] = [SPREAD[s]/(1-RECOV) for s in STOCKS]
EAD, PD = book['EAD weight'].values, book['PD (CDS-implied)'].values
EL, pdbar = (EAD*LGD*PD).sum(), (EAD*PD).sum()

disp = book.copy()
disp['EAD weight'] = disp['EAD weight'].map('{:.1%}'.format)
disp['PD (CDS-implied)'] = disp['PD (CDS-implied)'].map('{:.3%}'.format)
print(disp.to_string())
print(f'Expected loss: {EL*100:.3f}% of book | EAD-weighted avg PD: {pdbar*100:.2f}%')

def simulate_portfolio_loss(PD, EAD, LGD, rho, n=500_000, seed=42, matrix=False):
    g = np.random.default_rng(seed)
    A = np.sqrt(rho)*g.standard_normal((n, 1)) + np.sqrt(1-rho)*g.standard_normal((n, len(PD)))
    Dmat = A < norm.ppf(PD)
    return Dmat if matrix else (Dmat * EAD * LGD).sum(axis=1)

def vasicek_var(conf, PD, LGD, rho):
    num = norm.ppf(PD) + np.sqrt(rho) * norm.ppf(conf)
    cond_pd = norm.cdf(num / np.sqrt(1 - rho))
    return cond_pd * LGD

RHOS = [0.05, 0.15, 0.30]
print('\n3-name concentrated book. Credit VaR nets off expected loss, and')
print('economic capital is the 99.9% Credit VaR:')
print(f" {rho:>5}{VaR99'>9}{VaR99.9'>9}{CVaR99'>9}{CVaR99.9'>10}{ES99'>8}{ES99.9'>9}")
loss_show = None
for rho in RHOS:
    L = simulate_portfolio_loss(PD, EAD, LGD, rho)
    v99, v999 = np.percentile(L, 99), np.percentile(L, 99.9)
    print(f" {rho:>5.2f}{v99*100:8.2f}%{v999*100:8.2f}%{(v99-EL)*100:8.2f}%{(v999-EL)*100:9.2f}%")
    f'{L[L>=v99].mean()*100:7.2f}%{L[L>=v999].mean()*100:8.2f}%')
    if rho == 0.15: loss_show = L

print('\nEvery possible outcome of a 3-name book (loss atoms and their probability %):')
hdr = f" {defaults':<16}{loss':>8}" + ''.join(f'{"rho=">12}' for r in RHOS)
print(hdr); print(' ' + '-'*(len(hdr)-2))
Dmats = {rho: simulate_portfolio_loss(PD, EAD, LGD, rho, n=2_000_000, seed=1, matrix=True)
          for rho in RHOS}
atoms = []
for combo in itertools.product([0, 1], repeat=3):
    nm = ' + '.join(s for s, c in zip(STOCKS, combo) if c) or 'none'
    atoms.append((nm, np.array(combo), (np.array(combo)*EAD*LGD).sum()))
for nm, combo, loss in sorted(atoms, key=lambda a: -a[2]):
    ps = [np.all(Dmats[rho] == combo, axis=1).mean()*100 for rho in RHOS]
    print(f" {nm:<16}{loss*100:7.2f}% + ''.join(f'{p:>11.3f}' for p in ps))

print('\nGranular homogeneous pool (N=1000 at avg PD) vs Vasicek analytic:')
print(f" {rho:>4} {sim VaR99.9'>13} {Vasicek99.9'>12}")
PDp, EADp = np.full(1000, pdbar), np.full(1000, 1/1000)
for rho in RHOS:
    L = simulate_portfolio_loss(PDp, EADp, LGD, rho, n=200_000)
    print(f" {rho:>4.2f} {np.percentile(L, 99.9)*100:12.2f}%
    {vasicek_var(0.999, pdbar, LGD, rho)*100:11.2f}%")

print('\nThe same analytic ASRF capital, name by name on my actual three, against')
print('the simulated 3-name economic capital (both 99.9%, net of expected loss):')
print(f" {rho:>4} {sim EC (3 names)'>17} {ASRF EC (3 names)'>18}")
for rho in RHOS:
    L = simulate_portfolio_loss(PD, EAD, LGD, rho)
    sim_ec = np.percentile(L, 99.9) - EL
    asrf_ec = sum((vasicek_var(0.999, PD[i], LGD, rho) - LGD*PD[i]) * EAD[i]
                  for i in range(len(PD)))
    print(f" {rho:>4.2f} {sim_ec*100:>16.2f}% {asrf_ec*100:>17.2f}%")

fig, ax = plt.subplots(figsize=(11, 3.3))
ax.hist(loss_show*100, bins=60, color=BLUE, alpha=.7)
ax.axvline(EL*100, color=GREEN, lw=2, label=f'EL {EL*100:.2f}%')
ax.axvline(np.percentile(loss_show, 99.9)*100, color=RED, lw=2, label='CVaR 99.9%')
ax.set_yscale('log'); ax.set_xlabel('portfolio loss (% of book)')
ax.set_title('3-name loan-book loss distribution (rho = 0.15)'); ax.legend()
plt.tight_layout(); plt.show()

```

EAD weight PD (CDS-implied)

AAPL	43.0%	0.778%
PFE	32.8%	1.556%
XOM	24.2%	1.000%

Expected loss: 0.489% of book | EAD-weighted avg PD: 1.09%

3-name concentrated book. Credit VaR nets off expected loss, and economic capital is the 99.9% Credit VaR:

rho	VaR99	VaR99.9	CVaR99	CVaR99.9	ES99	ES99.9
0.05	14.78%	19.34%	14.29%	18.85%	16.62%	20.02%
0.15	14.78%	19.34%	14.29%	18.85%	16.82%	20.48%
0.30	14.78%	30.22%	14.29%	29.73%	17.35%	33.39%

Every possible outcome of a 3-name book (loss atoms and their probability %):

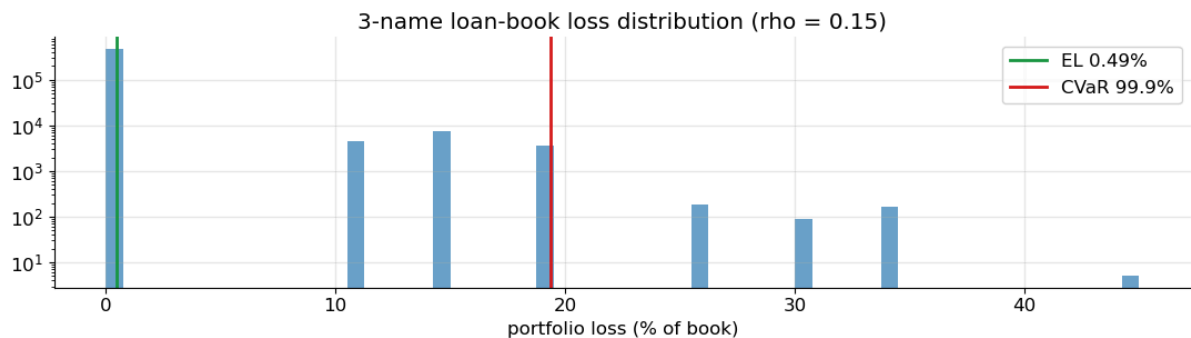
defaults	loss	rho=0.05	rho=0.15	rho=0.3
AAPL + PFE + XOM	45.00%	0.000%	0.002%	0.008%
AAPL + PFE	34.12%	0.017%	0.029%	0.056%
AAPL + XOM	30.22%	0.011%	0.019%	0.039%
PFE + XOM	25.66%	0.019%	0.036%	0.073%
AAPL	19.34%	0.747%	0.722%	0.670%
PFE	14.78%	1.517%	1.487%	1.411%
XOM	10.88%	0.964%	0.944%	0.880%
none	0.00%	96.726%	96.760%	96.862%

Granular homogeneous pool (N=1000 at avg PD) vs Vasicek analytic:

rho	sim VaR99.9	Vasicek99.9
0.05	2.39%	2.25%
0.15	5.45%	5.26%
0.30	10.89%	10.61%

The same analytic ASRF capital, name by name on my actual three, against the simulated 3-name economic capital (both 99.9%, net of expected loss):

rho	sim EC (3 names)	ASRF EC (3 names)
0.05	18.85%	1.74%
0.15	18.85%	4.70%
0.30	29.73%	9.96%



Interpretation

Expected loss is tiny at 0.489% of the book but the tail is brutal, and the reason is concentration. One PFE default alone costs 14.78%, so the 99% VaR sits there, a 14.29% Credit VaR net of expected loss. Economic capital, the 99.9% Credit VaR, runs from 18.85% to 29.73%, and ES is blunter, the rho = 0.30 99.9% ES at 33.39% against a 30.22% VaR. A thousand-borrower pool tracks Vasicek closely (2.39/5.45/10.89% against 2.25/5.26/10.61%), but run name by name on my actual three that same analytic formula asks only 4.70% of capital at rho = 0.15 against the 18.85% my simulation needs, and that gap is the concentration premium the granular IRB charge cannot see.

The surprise is the 99% VaR frozen at 14.78% for every correlation. Three names make only eight atoms, so PFE alone holds my whole 1% tail whatever correlation does. Raising rho from 0.05 to 0.30 barely moves the single-default atoms (PFE 1.517% to 1.411%) but more than triples the joint ones (AAPL and PFE 0.017% to 0.056%), so

once that mass clears 0.1% the 99.9% quantile leaves AAPL alone at 19.34% and jumps to AAPL and XOM at 30.22%.

The Gaussian factor may be too thin in a crisis, so I test it, keeping the PDs and correlations and swapping only the common factor for a Student-t with 4 degrees of freedom, plus a rho of 0.60. I also run the book on the structural PDs, floored at the rating table where Merton is effectively zero.

```
In [15]: NU_T, CRISIS_RHO = 4, 0.60

def simulate_portfolio_loss_t(PD, EAD, LGD, rho, nu=NU_T, n=500_000, seed=42, matrix=False):
    g = np.random.default_rng(seed)
    Z = np.sqrt(rho)*g.standard_normal((n, 1)) + np.sqrt(1-rho)*g.standard_normal((n, len(PD)))
    A = Z * np.sqrt(nu / g.chisquare(nu, (n, 1)))
    Dmat = A < stats.t.ppf(PD, nu)
    return Dmat if matrix else (Dmat * EAD * LGD).sum(axis=1)

print(f'Same PDs, same correlation, one factor swapped for a Student-t with nu = {NU_T}:')
print(f"{'rho':>6}{'P(2+ defaults) gauss':>22}{'t':>10}{'ratio':>8}"
      f"{'VaR99.9 gauss':>15}{'VaR99.9 t':>11}")
for rho in RHOS + [CRISIS_RHO]:
    d_gauss = simulate_portfolio_loss_t(PD, EAD, LGD, rho, n=2_000_000, seed=1, matrix=True)
    d_t = simulate_portfolio_loss_t(PD, EAD, LGD, rho, n=2_000_000, seed=1, matrix=True)
    p_gauss, p_t = (d_gauss.sum(1) >= 2).mean(), (d_t.sum(1) >= 2).mean()
    L_gauss, L_t = (d_gauss * EAD * LGD).sum(1), (d_t * EAD * LGD).sum(1)
    tag = ' <- crisis' if rho == CRISIS_RHO else ''
    print(f"{'rho':>6.2f}{'p_gauss*100:>21.3f'}%{'p_t*100:>9.3f'}%{'p_t/p_gauss:>7.1f'}x"
          f"{'np.percentile(L_gauss,99.9)*100:>14.2f'}%{'np.percentile(L_t,99.9)*100:>10.2f'}%"
          {tag})

pd_floored = np.array([max(merton_df.loc[s, 'PD Merton'], RATING_PD[RATING[s]]) for s in
STOCKS])
print('\nThe same book run on the structural PDs, floored at the rating table where')
print('Merton returns effectively zero (' + ', '.join(f'{s} {p:.2%}' for s, p
in zip(STOCKS, pd_floored)) + '):')
print(f"{'rho':>6}{'EL':>10}{'P(any default)':>17}{'VaR99':>9}{'VaR99.9':>10}")
for rho in RHOS:
    L = simulate_portfolio_loss(pd_floored, EAD, LGD, rho, n=2_000_000, seed=42)
    print(f"{'rho':>6.2f}{'(EAD*LGD*pd_floored).sum()*100:>9.3f'}%{'(L>0).mean()*100:>16.3f'}%"
          f"{'np.percentile(L,99)*100:>8.2f'}%{'np.percentile(L,99.9)*100:>9.2f'}%")
```

Same PDs, same correlation, one factor swapped for a Student-t with nu = 4:

rho	P(2+ defaults) gauss	t	ratio	VaR99.9 gauss	VaR99.9 t
0.05	0.047%	0.306%	6.5x	19.34%	34.12%
0.15	0.086%	0.364%	4.2x	19.34%	34.12%
0.30	0.176%	0.468%	2.7x	30.22%	34.12%
0.60	0.467%	0.696%	1.5x	34.12%	45.00% <- crisis

The same book run on the structural PDs, floored at the rating table where Merton returns effectively zero (AAPL 0.02%, PFE 0.05%, XOM 0.02%):

rho	EL	P(any default)	VaR99	VaR99.9
0.05	0.013%	0.089%	0.00%	0.00%
0.15	0.013%	0.088%	0.00%	0.00%
0.30	0.013%	0.090%	0.00%	0.00%

Interpretation

Swapping the factor does more than raising correlation. At rho = 0.05 the t makes two or more defaults 6.5 times more likely than the Gaussian, and my 99.9% VaR jumps from 19.34% to 34.12%, the AAPL and PFE atom the Gaussian only reaches at rho = 0.30. So a fatter tail buys more capital than tripling correlation.

The surprise is the ratio shrinking as correlation rises, from 6.5x at rho = 0.05 to 1.5x at 0.60. The t adds most where the Gaussian is most confident, so the two are substitutes not additions, and stressing only rho would have said the copula hardly matters. At rho = 0.60 the t still finds the all-three atom at 45.00%.

On the floored structural PDs my book expects to lose 0.013% and sees any default just 0.089% of the time, so both VaRs are zero, the quantile blind to anything rarer than one in a thousand. Same book and copula, my capital is either nothing or a third of the book depending only on which PD I believe.

3 · Derivatives, pricing, Greeks & VaR

I shock the spot only and hold time fixed, so the VaR isolates what delta and gamma describe.

```
In [16]: def bs_d1(S, K, T, r, sigma):
    return (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))

def bs_d2(S, K, T, r, sigma):
    return bs_d1(S, K, T, r, sigma) - sigma * np.sqrt(T)

def bs_call(S, K, T, r, sigma):
    d1, d2 = bs_d1(S, K, T, r, sigma), bs_d2(S, K, T, r, sigma)
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

def bs_delta(S, K, T, r, sigma, option='call'):
    d1 = bs_d1(S, K, T, r, sigma)
    if option == 'call':
        return norm.cdf(d1)
    return norm.cdf(d1) - 1

def bs_gamma(S, K, T, r, sigma):
    d1 = bs_d1(S, K, T, r, sigma)
    return norm.pdf(d1) / (S * sigma * np.sqrt(T))

def bs_vega(S, K, T, r, sigma):
    d1 = bs_d1(S, K, T, r, sigma)
    return S * norm.pdf(d1) * np.sqrt(T) / 100

def bs_theta(S, K, T, r, sigma, option='call'):
    d1 = bs_d1(S, K, T, r, sigma)
    d2 = bs_d2(S, K, T, r, sigma)
    term1 = -S * norm.pdf(d1) * sigma / (2 * np.sqrt(T))
    if option == 'call':
        return (term1 - r * K * np.exp(-r * T) * norm.cdf(d2)) / 365
    return (term1 + r * K * np.exp(-r * T) * norm.cdf(-d2)) / 365

def crr_price(S0, K, T, r, sigma, N, option='call', style='european'):
    dt = T / N
    u = np.exp(sigma * np.sqrt(dt))
    d = 1.0 / u
    p = (np.exp(r * dt) - d) / (u - d)
    disc = np.exp(-r * dt)

    j = np.arange(N + 1)
    ST = S0 * u**j * d**(N - j)

    if option == 'call':
        V = np.maximum(ST - K, 0)
    else:
        V = np.maximum(K - ST, 0)

    for i in range(N - 1, -1, -1):
        V = disc * (p * V[1:] + (1 - p) * V[:-1])
        if style == 'american':
            S_i = S0 * u**np.arange(i + 1) * d**(i - np.arange(i + 1))
            if option == 'call':
                V = np.maximum(V, S_i - K)
            else:
                V = np.maximum(V, K - S_i)
    return V[0]

S0, K, T_OPT, SIG = PRICE['AAPL'], PRICE['AAPL'], 0.25, SIGMA_E['AAPL']
call_bs = bs_call(S0, K, T_OPT, RF, SIG)
print(f'3-month ATM AAPL call | S0 = K = ${S0:.2f}, sigma = {SIG:.1%}, r = {RF:.0%}')
print(f'Black-Scholes: ${call_bs:.4f}\n')
print('CRR binomial convergence to Black-Scholes:')
Ns = [5, 10, 25, 50, 100, 250, 500, 2000]
crr = [crr_price(S0, K, T_OPT, RF, SIG, N) for N in Ns]
```

```

for N, c in zip(Ns, crr):
    print(f' N={N:>5}    CRR ${c:8.4f}    error {c-call_bs:+8.4f}')

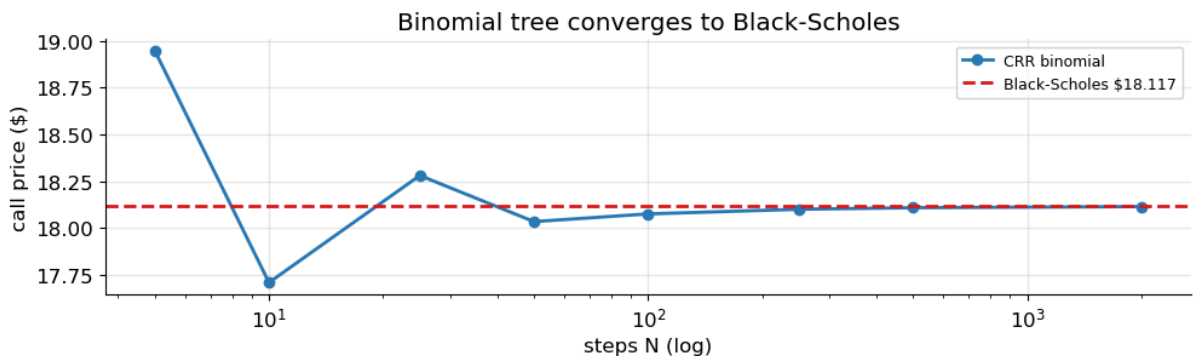
fig, ax = plt.subplots(figsize=(10, 3.2))
ax.plot(Ns, crr, 'o-', color=BLUE, lw=2, label='CRR binomial')
ax.axhline(call_bs, color=RED, ls='--', lw=2, label=f'Black-Scholes ${call_bs:.3f}')
ax.set_xscale('log'); ax.set_xlabel('steps N (log)'); ax.set_ylabel('call price ($)')
ax.set_title('Binomial tree converges to Black-Scholes'); ax.legend(fontsize=9)
plt.tight_layout(); plt.show()

```

3-month ATM AAPL call | $S_0 = K = \$317.31$, $\sigma = 26.2\%$, $r = 4\%$
 Black-Scholes: \$18.1170

CRR binomial convergence to Black-Scholes:

N= 5	CRR \$ 18.9468	error +0.8298
N= 10	CRR \$ 17.7093	error -0.4077
N= 25	CRR \$ 18.2807	error +0.1638
N= 50	CRR \$ 18.0345	error -0.0825
N= 100	CRR \$ 18.0757	error -0.0413
N= 250	CRR \$ 18.1004	error -0.0165
N= 500	CRR \$ 18.1087	error -0.0083
N= 2000	CRR \$ 18.1149	error -0.0021



```

In [17]: delta = bs_delta(S0, K, T_OPT, RF, SIG)
gamma = bs_gamma(S0, K, T_OPT, RF, SIG)
vega = bs_vega(S0, K, T_OPT, RF, SIG)
theta = bs_theta(S0, K, T_OPT, RF, SIG)
print(f'Greeks: delta {delta:.4f}    gamma {gamma:.5f}    vega ${vega:.4f} per +1% vol    '
      f'theta ${theta:.4f} per day')

bump_S, bump_sig = S0*1e-4, 1e-4
fd_delta = (bs_call(S0+bump_S, K, T_OPT, RF, SIG) - bs_call(S0-bump_S, K, T_OPT, RF, SIG)) /
(2*bump_S)
fd_gamma = (bs_call(S0+bump_S, K, T_OPT, RF, SIG) - 2*call_bs
            + bs_call(S0-bump_S, K, T_OPT, RF, SIG)) / bump_S**2
fd_vega = (bs_call(S0, K, T_OPT, RF, SIG+bump_sig)
            - bs_call(S0, K, T_OPT, RF, SIG-bump_sig)) / (2*bump_sig) / 100
print('\nClosed-form Greeks against bumping the pricer numerically:')
for nm, closed, bumped in [('delta', delta, fd_delta), ('gamma', gamma, fd_gamma),
                           ('vega', vega, fd_vega)]:
    print(f' {nm:<6} closed form {closed:>10.6f}    bumped {bumped:>10.6f}    '
          f'difference {closed-bumped:+.1e}')

g = np.random.default_rng(7)
var99 = lambda x: -np.percentile(x, 1)
print(f'\n{horizon:>8} {position:>6} {full_reval:>11} {delta-gamma:>12} "'
      f'{"delta-only:>11} {"delta-only error:>17} "')
for H, lab in [(1, '1-day'), (10, '10-day')]:
    dS = S0 * g.normal(0, SIG*np.sqrt(H/252), 400_000)
    pnl_full = bs_call(S0+dS, K, T_OPT, RF, SIG) - call_bs
    pnl_dg = delta*dS + 0.5*gamma*dS**2
    pnl_do = delta*dS
    for sign, nm in [(1, 'long'), (-1, 'short')]:
        f, dg, do = var99(sign*pnl_full), var99(sign*pnl_dg), var99(sign*pnl_do)
        print(f'{lab:>8} {nm:>6} ${f:>10.2f} ${dg:>11.2f} ${do:>10.2f} {(do-f)/f:>16.1%}')
    if H == 1: pnl_1d = pnl_full

fig, ax = plt.subplots(1, 2, figsize=(11, 3.3))
Sg = np.linspace(S0*.8, S0*1.2, 120)
ax[0].plot(Sg, bs_call(Sg, K, T_OPT, RF, SIG), color=BLUE, lw=2, label='full reval')
ax[0].plot(Sg, call_bs + delta*(Sg-S0), color=GREEN, ls='--', label='delta only')
ax[0].plot(Sg, call_bs + delta*(Sg-S0) + 0.5*gamma*(Sg-S0)**2, color=RED, ls=':', label='delta-
gamma')
ax[0].axvline(S0, color='gray', lw=.8); ax[0].set_title('Call value vs spot');
ax[0].legend(fontsize=9)
ax[1].hist(pnl_1d, bins=80, color=BLUE, alpha=.7)
ax[1].axvline(-var99(pnl_1d), color=RED, lw=2, label='99% VaR')

```

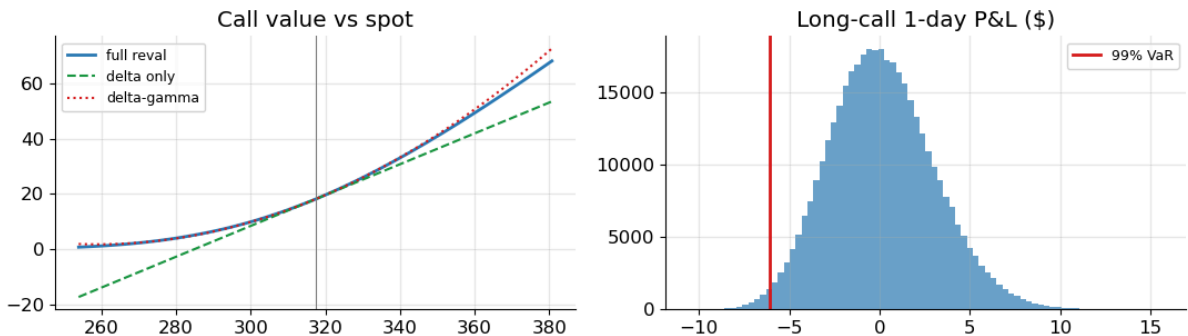
```
ax[1].set_title('Long-call 1-day P&L ($)'); ax[1].legend(fontsize=9)
plt.tight_layout(); plt.show()
```

Greeks: delta 0.5564 gamma 0.00950 vega \$0.6266 per +1% vol theta \$-0.1073 per day

Closed-form Greeks against bumping the pricer numerically:

delta	closed form	0.556395	bumped	0.556395	difference	+1.0e-08
gamma	closed form	0.009500	bumped	0.009500	difference	+4.3e-10
vega	closed form	0.626607	bumped	0.626607	difference	+3.3e-10

horizon	position	full reval	delta-gamma	delta-only	delta-only error
1-day	long	\$ 6.06	\$ 6.08	\$ 6.78	11.9%
1-day	short	\$ 7.48	\$ 7.50	\$ 6.79	-9.2%
10-day	long	\$ 14.29	\$ 14.37	\$ 21.39	49.6%
10-day	short	\$ 27.50	\$ 28.42	\$ 21.40	-22.2%



Interpretation

The tree converges onto Black-Scholes, \$18.95 at 5 steps down to \$18.1149 at 2000 against the closed form's \$18.1170, the error shrinking like $1/N$. Bumping the pricer returns the same delta, gamma and vega to eight decimals, so my formulas differentiate what I price, not whether it is right.

The option carries delta 0.556, gamma 0.0095 and vega \$0.63 per point of vol, and gamma is the whole story for VaR. Long the call, delta-only reports a 1-day 99% VaR of \$6.78 against a true \$6.06, overstating by 12% because positive gamma means it loses less than straight-line delta. Short it flips, a true \$7.48 while delta-only says \$6.79, understating by 9%, and delta-gamma catches both at \$6.08 and \$7.50. The same model is conservative for the buyer and dangerously optimistic for the seller.

Over 10 days delta-only overstates the long by 50% and understates the short by 22%, and even delta-gamma drifts, \$28.42 against \$27.50. Gamma is not free, theta is -\$0.11 a day, so the curvature flattering my long VaR is rented, paid every day the spot sits still.

4 · Liquidity-adjusted VaR

A liquidity cost can come off a quoted spread or off my own price and volume data, so I build both and compare.

```
In [18]: oh = pd.read_csv('A2/a2_ohlcv.csv', index_col=0, parse_dates=True)
close = oh[['close_{s}' for s in STOCKS]]; close.columns = STOCKS
volume = oh[['volume_{s}' for s in STOCKS]]; volume.columns = STOCKS
high = oh[['high_{s}' for s in STOCKS]]; high.columns = STOCKS
low = oh[['low_{s}' for s in STOCKS]]; low.columns = STOCKS
ret = np.log(close/close.shift(1))

dollar_vol = (close * volume).replace(0, np.nan)
amihud = (ret.abs() / dollar_vol).mean() * 1e9
adv = dollar_vol.mean()
```

```

def corwin_schultz_spread(high_s, low_s):
    h, l = high_s.values.astype(float), low_s.values.astype(float)
    h2, l2 = np.maximum(h[:-1], h[1:]), np.minimum(l[:-1], l[1:])
    beta = (np.log(h[:-1]/l[:-1]))**2 + (np.log(h[1:]/l[1:]))**2
    gamma = (np.log(h2/l2))**2
    denom = 3 - 2*np.sqrt(2)
    alpha = (np.sqrt(2*beta) - np.sqrt(beta))/denom - np.sqrt(gamma/denom)
    spread = 2*(np.exp(alpha) - 1) / (1 + np.exp(alpha))
    return pd.Series(np.where(alpha < 0, 0.0, spread), index=high_s.index[1:]), alpha

cs_df, cs_alpha = {}, {}
for s in STOCKS:
    cs_df[s], cs_alpha[s] = corwin_schultz_spread(high[s], low[s])
cs_df = pd.DataFrame(cs_df)

HALF_SPREAD = {'AAPL': 0.0001/2, 'PFE': 0.0006/2, 'XOM': 0.0003/2}

liq = pd.DataFrame({'Amihud (x1e9)': [f'{amihud[s]:.4f}' for s in STOCKS],
                    'Avg $vol (bn)': [f'{adv[s]/1e9:.2f}' for s in STOCKS],
                    'CS spread (bp)': [f'{cs_df[s].mean()*1e4:.1f}' for s in STOCKS],
                    'CS median (bp)': [f'{cs_df[s].median()*1e4:.1f}' for s in STOCKS],
                    'CS days floored at 0': [f'{(cs_alpha[s] < 0).mean():.0%}' for s in STOCKS],
                    'Quoted half-spread (bp)': [f'{HALF_SPREAD[s]*1e4:.1f}' for s in STOCKS]},
                    index=STOCKS)

print(liq.to_string())
print('\nSanity check: a real AAPL quoted spread is well under 1bp, so the CS mean of '
      f'{cs_df["AAPL"].mean()*1e4:.0f}bp is far too wide and CS does not rank the three names.')
print('I keep CS as a diagnostic and charge liquidity off Amihud instead.')

illiq_raw = (ret.abs() / dollar_vol).mean()

def amihud_liq_cost(book_value):
    return book_value * sum(W[i]**2 * illiq_raw[s] for i, s in enumerate(STOCKS))

BOOK = 1_000_000_000
spread_cost = sum(W[i]*HALF_SPREAD[s] for i, s in enumerate(STOCKS))
amihud_cost = amihud_liq_cost(BOOK)
LVAR_SPREAD = var_base + spread_cost
LVAR = var_base + amihud_cost

print(f'\nOn a ${BOOK/1e9:.0f}bn book:')
print(f'   Base VaR: {var_base*100:.2f}%')
print(f'   Liquidity-adjusted VaR (my own Amihud): {LVAR*100:.2f}% (cost {amihud_cost*100:.3f}%)')
print(f'   Liquidity-adjusted VaR (quoted spread): {LVAR_SPREAD*100:.2f}% (cost {spread_cost*100:.3f}%)')

print('\nAmihud is mean |return| per dollar of volume, so it scales with the position')
print('while a quoted spread does not:')
print(f"   {'book':>9}{ 'Amihud cost':>13}{ 'LVaR':>8}{ 'quoted-spread LVaR':>21}")
for PORT in [10e6, 100e6, BOOK]:
    c = amihud_liq_cost(PORT)
    print(f'   ${PORT/1e6:>7.0f}M{c*100:>12.3f}%{(var_base+c)*100:>7.2f}%{LVAR_SPREAD*100:>20.2f}%')

fig, ax = plt.subplots(1, 2, figsize=(11, 3.2))
ax[0].bar(STOCKS, [amihud[s] for s in STOCKS], color=RED)
ax[0].set_title('Amihud illiquidity (higher = less liquid)')
ax[1].bar(['base VaR', 'LVaR (Amihud)', 'LVaR (spread)'], [var_base*100, LVAR*100, LVAR_SPREAD*100],
          color=[BLUE, GREEN, RED])
ax[1].set_title(f'VaR vs liquidity-adjusted, ${BOOK/1e9:.0f}bn book (%)')
plt.tight_layout(); plt.show()

```

read (bp)	Amihud (x1e9)	Avg \$vol (bn)	CS spread (bp)	CS median (bp)	CS days floored at 0	Quoted half-spread
AAPL	0.0009	11.95	43.2	10.3		47%
0.5						
PFE	0.0110	1.04	46.7	17.7		45%
3.0						
XOM	0.0058	1.96	42.0	0.0		51%
1.5						

Sanity check: a real AAPL quoted spread is well under 1bp, so the CS mean of 43bp is far too wide and CS does not rank the three names.

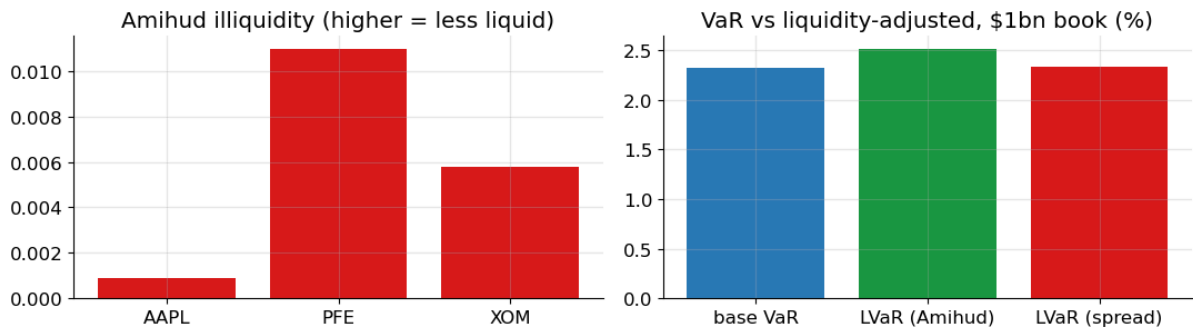
I keep CS as a diagnostic and charge liquidity off Amihud instead.

On a \$1bn book:

Base VaR: 2.32%
 Liquidity-adjusted VaR (my own Amihud): 2.52% (cost 0.196%)
 Liquidity-adjusted VaR (quoted spread): 2.34% (cost 0.017%)

Amihud is mean |return| per dollar of volume, so it scales with the position while a quoted spread does not:

	book	Amihud	cost	LVaR	quoted-spread	LVaR
\$	10M	0.002%	2.32%		2.34%	
\$	100M	0.020%	2.34%		2.34%	
\$	1000M	0.196%	2.52%		2.34%	



```
In [19]: BLOCK, PARTIC_MAX, ETA = BOOK, 0.20, 1.0
z_liq = stats.norm.ppf(0.99)
adv_252 = (close * volume).tail(252).mean()
REGIMES = [('normal', 1.0, 1.0, 1.0), ('stressed', 0.4, 2.0, 3.0)]

print(f'Unwinding the same ${BLOCK/1e6:.0f}M block in each name, never trading more than '
      f'${PARTIC_MAX:.0%} of a day\'s volume.')
print('The stressed regime cuts volume to 40% of normal, doubles daily volatility')
print('and triples the spread:\n')
hdr = (f'{"':>10} {'ADV ($bn/day)':>13} {'20% of ADV':>11} {'days':>5} {'half-spread':>12}
      f'{"impact':>10} {'timing risk':>12} {'total':>10} {'% of block':>10}")
print(hdr); print('-'*len(hdr))
endo = {}
for regime, vol_mult, sigma_mult, spread_mult in REGIMES:
    for s in STOCKS:
        sigma_d = ret[s].std() * sigma_mult
        hs = HALF_SPREAD[s] * spread_mult
        advd = adv_252[s] * vol_mult
        T = max(1.0, np.ceil(BLOCK / (PARTIC_MAX*advd)))
        x = (BLOCK/advd) / T
        spread_cost = BLOCK * hs
        impact = BLOCK * ETA * sigma_d * np.sqrt(x)
        timing = z_liq * sigma_d * BLOCK * np.sqrt(T/3.0)
        tot = spread_cost + impact + timing
        endo[(regime, s)] = dict(T=T, spread=spread_cost, impact=impact, timing=timing,
                                total=tot)
    print(f'{s+" "+regime[:4]:>10} ${advd/1e9:>12.2f} ${PARTIC_MAX*advd/1e6:>9.0f}M '
          f'{int(T):>5} ${spread_cost/1e6:>11.2f}M ${impact/1e6:>9.2f}M '
          f'${timing/1e6:>11.2f}M ${tot/1e6:>9.2f}M {tot/BLOCK:>9.2%}')
    print('-'*len(hdr))

n_pfe, n_aapl = endo[('normal', 'PFE')], endo[('normal', 'AAPL')]
s_pfe, s_aapl = endo[('stressed', 'PFE')], endo[('stressed', 'AAPL')]
print(f'\nNormally PFE takes {int(n_pfe['T'])} days to exit vs {int(n_aapl['T'])} for AAPL, and
costs "
      f"{n_pfe['total']/n_aapl['total']:.1f}x as much to get out of.")
print(f"The half-spread is {n_pfe['total']/n_pfe['spread']:.0f}x too small to describe "
      f"PFE's real exit cost.")
print(f"\nIn the stressed regime PFE takes {int(s_pfe['T'])} days and costs "
      f"{s_pfe['total']/BLOCK:.2%} of the block, {s_pfe['total']/n_pfe['total']:.1f}x the normal
cost, "
      f"while AAPL still clears in {int(s_aapl['T'])} day at {s_aapl['total']/BLOCK:.2%}.")
print(f"So the gap between my most and least liquid name widens from "
```

```

f"{n_pfe['total']/n_aapl['total']:.1f}x to {s_pfe['total']/s_aapl['total']:.1f}x "
f"exactly when I would be forced to sell.")

fig, ax = plt.subplots(figsize=(10, 3.2))
b = np.arange(len(STOCKS)); w = 0.38
for regime, off, alpha in [('normal', -w/2, 1.0), ('stressed', w/2, 0.55)]:
    sp = np.array([endo[(regime, s)]['spread']/1e6 for s in STOCKS])
    im = np.array([endo[(regime, s)]['impact']/1e6 for s in STOCKS])
    tm = np.array([endo[(regime, s)]['timing']/1e6 for s in STOCKS])
    lab = ['half-spread', 'market impact', 'timing risk (days to exit)'] if regime == 'normal' \
        else [None, None, None]
    ax.bar(b+off, sp, w, label=lab[0], color=GREEN, alpha=alpha)
    ax.bar(b+off, im, w, bottom=sp, label=lab[1], color=GOLD, alpha=alpha)
    ax.bar(b+off, tm, w, bottom=sp+im, label=lab[2], color=RED, alpha=alpha)
ax.set_xticks(b)
ax.set_xticklabels([f'{s}\n{int(endo[("normal", s)]["T"])}d / {int(endo[("stressed", s)]["T"])}d'
                    for s in STOCKS])
ax.set_ylabel('cost ($M)')
ax.set_title(f'Cost of unwinding a ${BLOCK/1e6:.0f}M block (left bar normal, right bar stressed)')
ax.legend(fontsize=9)
plt.tight_layout(); plt.show()

```

Unwinding the same \$1000M block in each name, never trading more than 20% of a day's volume. The stressed regime cuts volume to 40% of normal, doubles daily volatility and triples the spread:

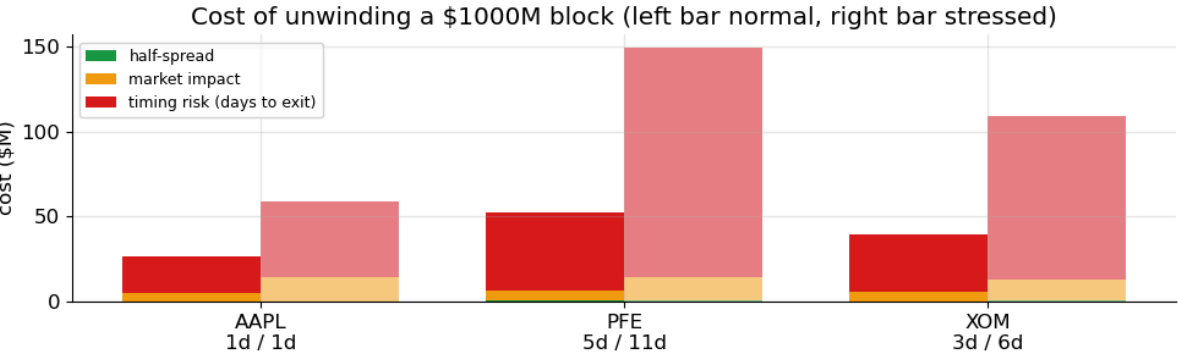
		ADV (\$bn/day)	20% of ADV	days	half-spread	impact	timing risk	total
% of block								

AAPL norm	\$	13.11	\$ 2622M	1	\$ 0.05M	\$ 4.56M	\$ 22.17M	\$ 26.78M
	2.68%							
PFE norm	\$	1.22	\$ 243M	5	\$ 0.30M	\$ 6.17M	\$ 45.71M	\$ 52.19M
	5.22%							
XOM norm	\$	2.33	\$ 465M	3	\$ 0.15M	\$ 5.55M	\$ 34.10M	\$ 39.80M
	3.98%							

AAPL stre	\$	5.24	\$ 1049M	1	\$ 0.15M	\$ 14.42M	\$ 44.34M	\$ 58.91M
	5.89%							
PFE stre	\$	0.49	\$ 97M	11	\$ 0.90M	\$ 13.16M	\$ 135.60M	\$ 149.66M
	14.97%							
XOM stre	\$	0.93	\$ 186M	6	\$ 0.45M	\$ 12.41M	\$ 96.46M	\$ 109.32M
	10.93%							

Normally PFE takes 5 days to exit vs 1 for AAPL, and costs 1.9x as much to get out of. The half-spread is 174x too small to describe PFE's real exit cost.

In the stressed regime PFE takes 11 days and costs 14.97% of the block, 2.9x the normal cost, while AAPL still clears in 1 day at 5.89%. So the gap between my most and least liquid name widens from 1.9x to 2.5x exactly when I would be forced to sell.



Interpretation

PFE is the least liquid, an Amihud ratio of 0.011 against AAPL's 0.0009. On a \$1bn book Amihud charges 0.196%, lifting my VaR from 2.32% to 2.52%, while a half-spread charges only 0.017% and says 2.34%. I take Amihud because it knows my size, the

same names costing 0.002% on a \$10M book and 0.196% on \$1bn, crossing near \$100M, so a spread charge fits one book size.

Corwin-Schultz failed outright, AAPL at 43bp against a real spread under 1bp because it reads the daily high-low range, which for tick-tight names is volatility and gaps not the bid-ask bounce. Half its days floor a negative alpha at zero, so AAPL's 10bp median against a 43bp mean is noise.

Unwinding a \$1bn block name by name is worse. AAPL is gone in a day, PFE takes 5 because 20% of its volume is only about \$243M, so PFE's \$45.7M timing risk and \$6.2M impact against a \$0.3M half-spread cost 5.22% of the block versus AAPL's 2.68%. That timing already sits in my base VaR. Under stress (volume at 40%, double volatility, triple spreads) PFE takes 11 days and 14.97% while AAPL clears in a day at 5.89%, so my easiest-to-hardest gap widens from 1.9x to 2.5x, the diversification I thought I had thinning out where I would lean on it.

5 · Operational-loss distribution

My parameters are judgemental. A large-cap non-financial faces a handful of material events a year across product liability, litigation, cyber and environmental, so I take 3 a year, a \$5M median event, and sigma 1.5 for a tail reaching the billions these firms operate in.

```
In [20]: LAMBDA, MU_SEV, SIG_SEV = 3.0, np.log(5e6), 1.5
g = np.random.default_rng(123); YEARS = 1_000_000
counts = g.poisson(LAMBDA, YEARS)
sev = g.lognormal(MU_SEV, SIG_SEV, counts.sum())
annual = np.bincount(np.repeat(np.arange(YEARS), counts), weights=sev, minlength=YEARS)

mean_sev = np.exp(MU_SEV + 0.5*SIG_SEV**2)
EL_OP = annual.mean()
OP99 = np.percentile(annual, 99)
OP999 = np.percentile(annual, 99.9)
print(f'Frequency: Poisson(lambda={LAMBDA}) | Severity: lognormal, median '
      f'${np.exp(MU_SEV)/1e6:.0f}M, mean ${mean_sev/1e6:.1f}M')
print(f'Expected annual op loss: ${EL_OP/1e6:.1f}M')
print(f'99% annual op loss:      ${OP99/1e6:.0f}M')
print(f'99.9% annual op loss:    ${OP999/1e6:.0f}M -> Op capital (99.9% - EL): '
      f'${(OP999-EL_OP)/1e6:.0f}M')

normal_999 = annual.mean() + norm.ppf(0.999)*annual.std()
print(f'\nShape of the annual loss distribution:')
print(f' skew {skew(annual):.2f} | excess kurtosis {kurtosis(annual):.1f} '
      f'(Normal = 0 for both)')
print(f' 99.9% / 99% ratio: {OP999/OP99:.2f}')
print(f' 99.9% / mean ratio: {OP999/EL_OP:.1f}')
print(f' Normal approx 99.9%: ${normal_999/1e6:.0f}M vs simulated ${OP999/1e6:.0f}M '
      f'-> a Normal understates the 99.9% loss by {100*(1-normal_999/OP999):.0f}%)')
print(f' and understates capital (99.9% - EL) by '
      f'{100*(1-(normal_999-EL_OP)/(OP999-EL_OP)):0f}%)')

BEXTRA = 2.3e9
pct_bextra = (annual < BEXTRA).mean()*100
print(f'\nWhere a real settlement lands in my distribution:')
print(f' Pfizer paid ${BEXTRA/1e9:.1f}bn over Bextra in 2009. That single event sits at my '
      f'{pct_bextra:.3f}th percentile,')
print(f' a 1-in-{1/(1-pct_bextra/100):.0f}-year total, against a worst simulated year of '
      f'${annual.max()/1e6:.0f}M in a million.')
print(f' My model gives a year that bad a {100-pct_bextra:.4f}% chance.')

fig, ax = plt.subplots(figsize=(11, 3.3))
ax.hist(annual/1e6, bins=120, color=BLUE, alpha=.7)
ax.axvline(EL_OP/1e6, color=GREEN, lw=2, label=f'EL ${EL_OP/1e6:.0f}M')
ax.axvline(OP999/1e6, color=RED, lw=2, label=f'99.9% ${OP999/1e6:.0f}M')
ax.axvline(BEXTRA/1e6, color=GOLD, lw=2, label=f'Bextra ${BEXTRA/1e9:.1f}bn')
```

```
ax.set_xlim(0, BEXTRA/1e6*1.1); ax.set_yscale('log')
ax.set_xlabel('annual operational loss ($M)')
ax.set_title('Simulated annual operational loss'); ax.legend()
plt.tight_layout(); plt.show()
```

Frequency: Poisson(lambda=3.0) | Severity: lognormal, median \$5M, mean \$15.4M

Expected annual op loss: \$46.1M

99% annual op loss: \$342M

99.9% annual op loss: \$876M -> Op capital (99.9% - EL): \$830M

Shape of the annual loss distribution:

skew 9.92 | excess kurtosis 234.5 (Normal = 0 for both)

99.9% / 99% ratio: 2.56

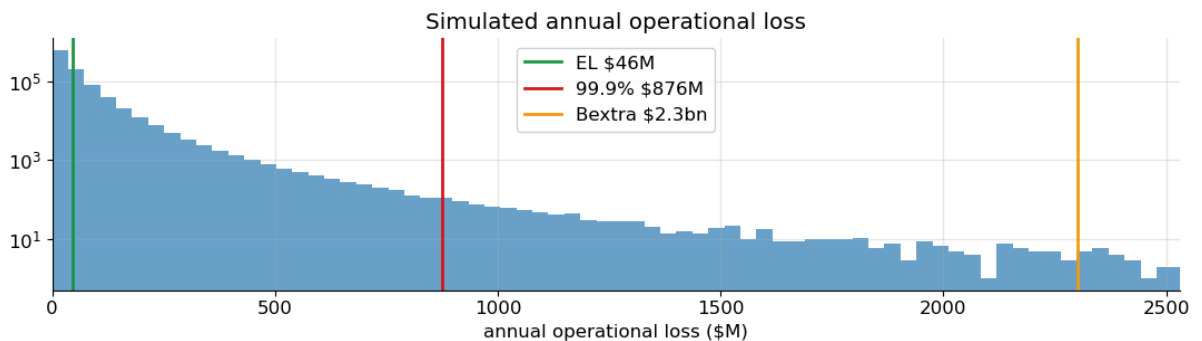
99.9% / mean ratio: 19.0

Normal approx 99.9%: \$294M vs simulated \$876M -> a Normal understates the 99.9% loss by 66% and understates capital (99.9% - EL) by 70%

Where a real settlement lands in my distribution:

Pfizer paid \$2.3bn over Bextra in 2009. That single event sits at my 99.993th percentile, a 1-in-13,889-year total, against a worst simulated year of \$4,310M in a million.

My model gives a year that bad a 0.0072% chance.



```
In [21]: def lda_capital(lam, mu, sg, years=YEARS, seed=123):
    gg = np.random.default_rng(seed)
    c = gg.poisson(lam, years)
    sv = gg.lognormal(mu, sg, c.sum())
    a = np.bincount(np.repeat(np.arange(years), c), weights=sv, minlength=years)
    return np.percentile(a, 99.9) - a.mean()

base = lda_capital(LAMBDA, MU_SEV, SIG_SEV)
print('Sensitivity of the 99.9% capital number to my assumptions:\n')
print(f" {'assumption':<28}{'capital':>12}{'vs base':>10}")
print(' ' + '-'*48)
for lam in [1.5, 3.0, 6.0]:
    c = lda_capital(lam, MU_SEV, SIG_SEV)
    print(f" {'lambda':<10}{'events/yr':<28}{'c/1e6':>10.0f}M{'c/base':>9.2f}x")
for sg in [1.0, 1.5, 2.0, 2.5]:
    c = lda_capital(LAMBDA, MU_SEV, sg)
    mean_sg = np.exp(MU_SEV + 0.5*sg**2)
    print(f" {'severity sigma':<28}{'c/1e6':>10.0f}M{'c/base':>9.2f}x"
          f" (mean event ${mean_sg/1e6:.1f}M)")

print('')
print('Raising sigma also raises the mean event, so the rows above mix a fatter tail')
print('with a bigger average loss. Holding the mean event fixed at $15.4M isolates')
print('the tail shape on its own:')
for sg in [1.0, 1.5, 2.0, 2.5]:
    mu_fixed = MU_SEV + 0.5*(SIG_SEV**2 - sg**2)
    c = lda_capital(LAMBDA, mu_fixed, sg)
    print(f" {'sigma':<10}{'(mean held)':<28}{'c/1e6':>10.0f}M{'c/base':>9.2f}x"
          f" (mean event ${np.exp(mu_fixed + 0.5*sg**2)/1e6:.1f}M)")

def business_indicator_component(BI):
    b1, b2 = 1e9, 30e9
    if BI <= b1: return 0.12*BI
    if BI <= b2: return 0.12*b1 + 0.15*(BI - b1)
    return 0.12*b1 + 0.15*(b2 - b1) + 0.18*(BI - b2)

def internal_loss_multiplier(LC, BIC):
    return np.log(np.e - 1 + (LC/BIC)**0.8)

BI = 20e9
LC = 15 * EL_OP
print('The SMA is a bank capital rule, so it does not apply to AAPL, PFE or XOM directly.')
print('I run it for a mid-size bank carrying a loss profile like theirs, as the')
print('standardized comparator to the internal model above.')
BIC = business_indicator_component(BI)
ILM = internal_loss_multiplier(LC, BIC)
```

```
print(f'\nBasel SMA (the rule since 2023), for a mid-size bank with BI = ${BI/1e9:.0f}bn:')
print(f' BIC ${BIC/1e9:.2f}bn x ILM {ILM:.3f} = SMA capital ${BIC*ILM/1e9:.2f}bn')
print(f' My 99.9% economic capital: ${((OP999-EL_OP)/1e9:.2f)bn '
      f'-> SMA charges {BIC*ILM/(OP999-EL_OP):.1f}x my internal model')
```

Sensitivity of the 99.9% capital number to my assumptions:

assumption	capital	vs base	
lambda = 1.5 events/yr	\$ 620M	0.75x	
lambda = 3.0 events/yr	\$ 830M	1.00x	
lambda = 6.0 events/yr	\$ 1092M	1.32x	
severity sigma = 1.0	\$ 161M	0.19x	(mean event \$8.2M)
severity sigma = 1.5	\$ 830M	1.00x	(mean event \$15.4M)
severity sigma = 2.0	\$ 4485M	5.40x	(mean event \$36.9M)
severity sigma = 2.5	\$ 24484M	29.50x	(mean event \$113.8M)

Raising sigma also raises the mean event, so the rows above mix a fatter tail with a bigger average loss. Holding the mean event fixed at \$15.4M isolates the tail shape on its own:

sigma = 1.0 (mean held)	\$ 301M	0.36x	(mean event \$15.4M)
sigma = 1.5 (mean held)	\$ 830M	1.00x	(mean event \$15.4M)
sigma = 2.0 (mean held)	\$ 1870M	2.25x	(mean event \$15.4M)
sigma = 2.5 (mean held)	\$ 3314M	3.99x	(mean event \$15.4M)

The SMA is a bank capital rule, so it does not apply to AAPL, PFE or XOM directly. I run it for a mid-size bank carrying a loss profile like theirs, as the standardized comparator to the internal model above.

Basel SMA (the rule since 2023), for a mid-size bank with BI = \$20bn:
 BIC \$2.97bn x ILM 0.708 = SMA capital \$2.10bn
 My 99.9% economic capital: \$0.83bn -> SMA charges 2.5x my internal model

Interpretation

My average year loses \$46.1M but the 99.9% year loses \$876M, so operational capital is about \$830M. Skew is 9.9 and excess kurtosis 234 against 0 for a Normal, and a Normal on the same mean and standard deviation puts the 99.9% loss at \$294M, understating it by 66% and the capital by 70%. It is not abstract, since Pfizer's \$2.3bn 2009 Bextra settlement lands at the 99.993th percentile, one in 13,889 years, so a loss already paid sits too far out, my sigma of 1.5 too thin.

The sensitivity is lopsided. Doubling frequency from 3 to 6 events raises capital 1.32 times, but severity sigma from 1.5 to 2.0 multiplies it 5.4 times and 2.5 multiplies it 29.5 times. That flatters my point, since raising sigma also lifts the mean event from \$15.4M to \$36.9M, but with the mean fixed capital still moves 2.25 times at 2.0 and 4.0 times at 2.5, tail shape beating frequency by three times not twenty because at the 99.9% quantile my year is basically one event. That least-evidenced parameter sets the answer, why Basel scrapped internal models here, and for a mid-size bank with these losses the SMA charges \$2.10bn against my \$0.83bn, reading my size, never my tail.

6 · Basel, credit, market & operational risk

It rests on expected versus unexpected loss. Expected loss is provisioned, so capital covers only the gap to a bad year, at a confidence level the regulator picks.

Credit is standardized weights off the rating or the IRB Vasicek charge my granular pool matched. It needs a large diversified pool, so my three-name book is the case it cannot see, where almost all risk is one borrower alone and one Gaussian correlation misses what a t factor did to my 99.9% loss.

Market risk gets standardized sensitivities or an internal FRTB track that measures Expected Shortfall not VaR, since a quantile is silent on what sits behind it, the blindness that froze my 99% number on one atom. Its liquidity horizons match my PFE at eleven days under stress, and Basel replaced operational internal models with the SMA my sensitivity table argues for.

My three differ. PFE pulls credit, liquidity and operational risk into one obligor an IRB charge for a diversified pool cannot see, AAPL's size makes its rare default my worst single loss, and XOM only moves my 99.9% once correlation reaches 0.30.

Each model ranked my firms sensibly but none on the level, the half-spread understating PFE's exit by 174 times, so I check the level against something outside the model.

Assignment 3: Portfolio Optimization, Stress Testing & Systemic Risk

AAPL, PFE and XOM against the S&P 500. The data is a dated snapshot and every simulation is seeded, so this reproduces. I use a 4% risk free rate throughout, close to the 3-month Treasury bill over these three years.

```
In [22]: import warnings
warnings.filterwarnings('ignore')
import numpy as np, pandas as pd, matplotlib.pyplot as plt
import statsmodels.api as sm
from numpy.linalg import inv
from scipy.optimize import minimize
from scipy.stats import spearmanr
from pypfport import EfficientFrontier, expected_returns, risk_models
from statsmodels.regression.quantile_regression import QuantReg
from IPython.display import display
np.random.seed(42)

plt.rcParams.update({'figure.figsize': (13, 5), 'axes.grid': True, 'grid.alpha': 0.3,
                    'axes.spines.top': False, 'axes.spines.right': False, 'font.size': 12})
BLUE, RED, GREEN, GOLD, GREY = '#2C7BB6', '#D7191C', '#1A9641', '#F39C12', '#6B7280'

STOCKS = ['AAPL', 'PFE', 'XOM']
BENCH = '^GSPC'
RF = 0.04

px = pd.read_csv('A1/my_portfolio.csv', index_col=0, parse_dates=True)
rets = px[STOCKS].pct_change().dropna()
mkt = px[BENCH].pct_change().dropna()

MARKET_CAP = {'AAPL': 4.660445e12, 'PFE': 1.395224e11, 'XOM': 5.989863e11}
w_mkt = (pd.Series(MARKET_CAP) / sum(MARKET_CAP.values()))[STOCKS]

mu = expected_returns.mean_historical_return(rets, returns_data=True, compounding=False)
S_sample = risk_models.sample_cov(rets, returns_data=True)
shrinkage = risk_models.CovarianceShrinkage(rets, returns_data=True)
S_lw = shrinkage.ledoit_wolf()

W_EQ = np.ones(3) / 3
port_eq = (rets * W_EQ).sum(axis=1)
var99 = -np.percentile(port_eq, 1)
es975 = -port_eq[port_eq <= np.percentile(port_eq, 2.5)].mean()

print(f'Sample {rets.index[0].date()} to {rets.index[-1].date()}, {len(rets)} trading days')
print(f"{'':6}{'ann return':>13}{'ann vol':>10}{'market cap':>14}{'cap weight':>12}")
for s in STOCKS:
    print(f'{s:6}{mu[s]*100:>12.2f}%{np.sqrt(S_sample.loc[s, s])*100:>9.2f}% '
          f'{MARKET_CAP[s]/1e12:>12.2f}tn{w_mkt[s]*100:>11.1f}%')
print(f'\nCorrelation matrix:\n{rets.corr().round(3).to_string()}')
print(f'\nEqual-weight book: ann vol {port_eq.std()*np.sqrt(252)*100:.2f}%, '
```

```
f'1-day 99% VaR {var99*100:.2f}%, 97.5% ES {es975*100:.2f}%')
print(f'Ledoit-Wolf shrinkage intensity {shrinkage.delta:.3f} '
      f'condition number: sample {np.linalg.cond(S_sample.values):.2f}, '
      f'shrunk {np.linalg.cond(S_lw.values):.2f}')
```

Sample 2023-07-05 to 2026-06-30, 750 trading days

	ann return	ann vol	market cap	cap weight
AAPL	17.61%	26.32%	4.66tn	86.3%
PFE	-4.92%	24.18%	0.14tn	2.6%
XOM	14.16%	23.21%	0.60tn	11.1%

Correlation matrix:

	AAPL	PFE	XOM
AAPL	1.000	0.181	0.11
PFE	0.181	1.000	0.13
XOM	0.110	0.130	1.00

Equal-weight book: ann vol 16.08%, 1-day 99% VaR 2.42%, 97.5% ES 2.89%

Ledoit-Wolf shrinkage intensity 0.319 condition number: sample 1.66, shrunk 1.42

1 · Efficient frontier, minimum variance, tangency and risk parity

Weights are long-only and fully invested, since I cannot borrow or short, and that constraint does a lot of the work below.

```
In [23]: def optimise(expected, covariance, objective='max_sharpe', target=None,
              risk_aversion=None, constraint=None):
    frontier = EfficientFrontier(expected, covariance, weight_bounds=(0, 1))
    if constraint is not None:
        frontier.add_constraint(constraint)
    if objective == 'max_sharpe':
        frontier.max_sharpe(risk_free_rate=RF)
    elif objective == 'min_volatility':
        frontier.min_volatility()
    elif objective == 'efficient_return':
        frontier.efficient_return(target_return=target)
    elif objective == 'max_quadratic_utility':
        frontier.max_quadratic_utility(risk_aversion=risk_aversion)
    cleaned = frontier.clean_weights()
    return np.array([cleaned[s] for s in STOCKS])

def port_return(w, m): return float(np.asarray(w) @ np.asarray(m))
def port_vol(w, S): return float(np.sqrt(np.asarray(w) @ np.asarray(S) @ np.asarray(w)))
def sharpe(w, m, S): return (port_return(w, m) - RF) / port_vol(w, S)

def risk_contributions(w, S):
    w = np.asarray(w, dtype=float)
    total = np.sqrt(w @ S @ w)
    return w * (S @ w / total)

def erc_objective(w, S):
    rc = risk_contributions(w, S)
    return np.sum((rc[:, None] - rc[None, :]) ** 2)

def equal_risk(S):
    result = minimize(erc_objective, np.ones(3) / 3, args=(S,), method='SLSQP',
                      bounds=[(0.0, 1.0)] * 3,
                      constraints=({'type': 'eq', 'fun': lambda w: w.sum() - 1.0,}),
                      options={'maxiter': 1000, 'ftol': 1e-12})
    return result.x

Sv, mv = S_sample.values, mu.values
w_minvar = optimise(mu, S_sample, 'min_volatility')
w_tan = optimise(mu, S_sample, 'max_sharpe')
w_erc = equal_risk(Sv)

base_portfolios = {'Equal weight': W_EQ, 'Minimum variance': w_minvar,
                  'Tangency (max Sharpe)': w_tan, 'Risk parity (ERC)': w_erc}

print(f'{"portfolio":<24}{"AAPL":>8}{"PFE":>8}{"XOM":>8}{"return":>9}{"vol":>8}{"Sharpe":>8}')
for name, w in base_portfolios.items():
    print(f'{name:<24}' + ''.join(f'{x*100:>7.1f}%' for x in w) +
          f'{port_return(w, mv)*100:>8.2f}%' + f'{port_vol(w, Sv)*100:>7.2f}%' + f'{sharpe(w, mv, Sv):>8.3f}%' )

print(f'\nRisk contributions as a share of portfolio volatility:')
```

```

print(f'{"portfolio":<24>{"AAPL":>8>{"PFE":>8>{"XOM":>8>{"spread":>10}"')
for name, w in base_portfolios.items():
    rc = risk_contributions(w, Sv); rc = rc / rc.sum()
    print(f'{name:<24}' + ''.join(f'{x*100:>7.1f}%' for x in rc) +
          f' {(rc.max()-rc.min())*100:>9.1f}pp')

targets = np.linspace(port_return(w_minvar, mv), mv.max() - 1e-6, 60)
frontier = [(port_vol(optimize(mu, S_sample, 'efficient_return', target=t), Sv), t)
             for t in targets]
fx, fy = zip(*frontier)

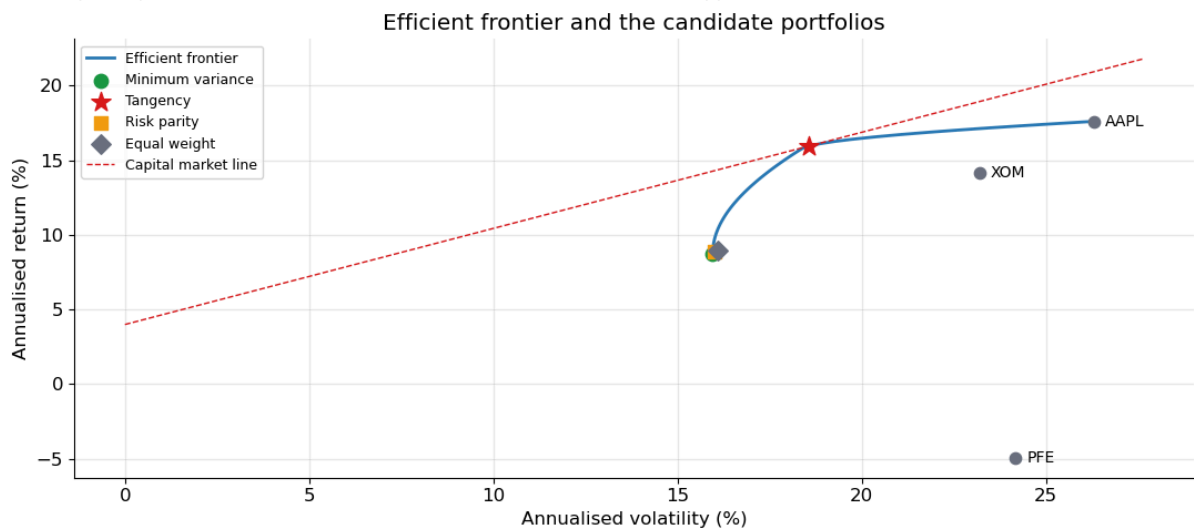
fig, ax = plt.subplots(figsize=(11, 5))
ax.plot(np.array(fx)*100, np.array(fy)*100, color=BLUE, lw=2, label='Efficient frontier')
for s in STOCKS:
    ax.scatter(np.sqrt(S_sample.loc[s, s])*100, mu[s]*100, color=GREY, s=55, zorder=3)
    ax.annotate(s, (np.sqrt(S_sample.loc[s, s])*100, mu[s]*100),
               textcoords='offset points', xytext=(7, -3), fontsize=10)
marks = [('Minimum variance', w_minvar, GREEN, 'o'), ('Tangency', w_tan, RED, '*'),
         ('Risk parity', w_erc, GOLD, 's'), ('Equal weight', w_EQ, GREY, 'D')]
for name, w, colour, marker in marks:
    ax.scatter(port_vol(w, Sv)*100, port_return(w, mv)*100, color=colour,
               s=170 if marker == '*' else 80, marker=marker, zorder=4, label=name)
cml_x = np.linspace(0, max(fx)*1.05, 50)
ax.plot(cml_x*100, (RF + sharpe(w_tan, mv, Sv)*cml_x)*100, color=RED, ls='--', lw=1,
        label='Capital market line')
ax.set_xlabel('Annualised volatility (%)'); ax.set_ylabel('Annualised return (%)')
ax.set_title('Efficient frontier and the candidate portfolios'); ax.legend(fontsize=9)
plt.tight_layout(); plt.show()

```

portfolio	AAPL	PFE	XOM	return	vol	Sharpe
Equal weight	33.3%	33.3%	33.3%	8.95%	16.08%	0.308
Minimum variance	27.4%	33.2%	39.4%	8.76%	15.96%	0.299
Tangency (max Sharpe)	52.0%	0.0%	48.0%	15.96%	18.58%	0.644
Risk parity (ERC)	30.9%	33.3%	35.8%	8.87%	16.00%	0.305

Risk contributions as a share of portfolio volatility:

portfolio	AAPL	PFE	XOM	spread
Equal weight	37.6%	33.2%	29.2%	8.5pp
Minimum variance	27.4%	33.2%	39.4%	12.0pp
Tangency (max Sharpe)	59.2%	0.0%	40.8%	59.2pp
Risk parity (ERC)	33.3%	33.3%	33.3%	0.0pp



Interpretation

My three names barely move together, with correlations of 0.181, 0.110 and 0.130. That pulls minimum variance to 15.96% volatility, below the 26.32%, 24.18% and 23.21% they run alone. Tangency drops PFE and splits 52.0% AAPL against 48.0% XOM, lifting Sharpe from 0.308 to 0.644. That is not skill. PFE lost 4.92% a year, and any optimiser handed a negative mean and a long-only constraint zeroes it.

Risk parity surprised me at 30.9/33.3/35.8, basically equal weight, with a 0.305 Sharpe against the 0.308 for splitting three ways. It is meant to stop the jumpiest stock dominating, but mine are all about as jumpy as each other. Tangency instead runs a

59.2pp risk-contribution spread with AAPL supplying 59.2% of the risk, so my best Sharpe is the least diversified thing on the page.

2 · Black-Litterman

Market caps are shares outstanding times the close on 30 June 2026, and risk aversion comes from my own cap-weighted portfolio rather than the textbook 2.5. My relative view gives AAPL 3 points a year over XOM, backing a platform earnings base over a cyclical energy margin, and the absolute view puts PFE at 5%, a partial recovery as its pipeline steadies rather than a repeat of its sample loss. The confidence on each is the proportional prior at $\tau = 0.05$, scaling with the volatility of the portfolio expressing it.

```
In [24]: TAU = 0.05
P = pd.DataFrame([[1, 0, -1], [0, 1, 0]], columns=STOCKS)
Q = pd.Series([0.03, 0.05])

r_mkt = (rets * w_mkt.values).sum(axis=1)
delta = (r_mkt.mean()*252 - RF) / (r_mkt.std()*2 * 252)

def implied_returns(risk_aversion, sigma, w):
    equilibrium = risk_aversion * sigma.dot(w).squeeze()
    equilibrium.name = 'implied'
    return equilibrium

def proportional_prior(sigma, tau, p):
    helit = p.dot(tau * sigma).dot(p.T)
    return pd.DataFrame(np.diag(np.diag(helit.values)), index=p.index, columns=p.index)

def black_litterman(w_prior, sigma_prior, p, q, omega, risk_aversion, tau):
    pi = implied_returns(risk_aversion, sigma_prior, w_prior)
    sps = tau * sigma_prior
    posterior_mu = pi + sps.dot(p.T).dot(
        inv(p.dot(sps).dot(p.T) + omega).dot(q - p.dot(pi).values))
    posterior_sigma = sigma_prior + sps - sps.dot(p.T).dot(
        inv(p.dot(sps).dot(p.T) + omega)).dot(p).dot(sps)
    return posterior_mu, posterior_sigma

omega = proportional_prior(S_lw, TAU, P)
pi = implied_returns(delta, S_lw, w_mkt)
mu_bl, S_bl = black_litterman(w_mkt, S_lw, P, Q, omega, delta, TAU)

w_bl = optimise(mu_bl, S_bl, 'max_quadratic_utility', risk_aversion=delta)

print(f'Cap-weighted portfolio: return {r_mkt.mean()*252*100:.2f}%, '
      f'vol {r_mkt.std()*np.sqrt(252)*100:.2f}% -> implied risk aversion delta = {delta:.2f}')
print(f'View uncertainty (standard deviation): '
      f'{", ".join(f"view {i+1} {np.sqrt(omega.values[i, i])*100:.1f}%" for i in range(2))}')
print(f"\n{'':6}{ 'cap weight':>12}{ 'equilibrium':>13}{ 'historical':>12}{ 'posterior':>11}")
for s in STOCKS:
    print(f'{s:6}{w_mkt[s]*100:>11.1f}%{pi[s]*100:>12.2f}%{mu[s]*100:>11.2f}%
          {mu_bl[s]*100:>10.2f}%')

print(f"\n{'portfolio':<24}{ 'AAPL':>8}{ 'PFE':>8}{ 'XOM':>8}{ 'return':>9}{ 'vol':>8}{ 'Sharpe':>8}")
for name, w in [( 'Market cap weights', w_mkt.values), ( 'Black-Litterman', w_bl),
                ( 'Tangency on history', w_tan)]:
    print(f'{name:<24}' + ''.join(f'{x*100:>7.1f}%' for x in w) +
          f'{port_return(w, mv)*100:>8.2f}%{port_vol(w, Sv)*100:>7.2f}%{sharpe(w, mv,
Sv):>8.3f}'))

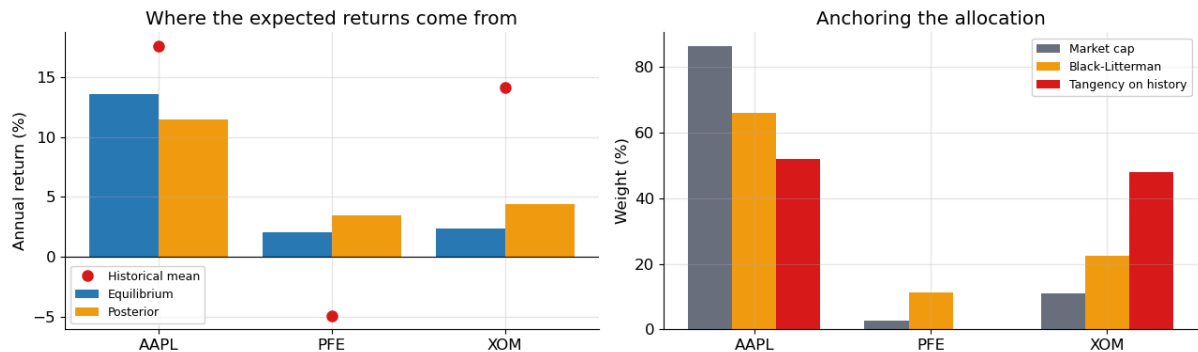
fig, (axL, axR) = plt.subplots(1, 2, figsize=(13, 4))
x = np.arange(3)
axL.bar(x - 0.2, pi.values*100, 0.4, color=BLUE, label='Equilibrium')
axL.bar(x + 0.2, mu_bl.values*100, 0.4, color=GOLD, label='Posterior')
axL.plot(x, mu.values*100, 'o', color=RED, ms=8, label='Historical mean')
axL.axhline(0, color='black', lw=0.8)
axL.set_xticks(x); axL.set_xticklabels(STOCKS); axL.set_ylabel('Annual return (%)')
axL.set_title('Where the expected returns come from'); axL.legend(fontsize=9)
axR.bar(x - 0.25, w_mkt.values*100, 0.25, color=GREY, label='Market cap')
axR.bar(x, w_bl*100, 0.25, color=GOLD, label='Black-Litterman')
axR.bar(x + 0.25, w_tan*100, 0.25, color=RED, label='Tangency on history')
axR.set_xticks(x); axR.set_xticklabels(STOCKS); axR.set_ylabel('Weight (%)')
```

```
axR.set_title('Anchoring the allocation'); axR.legend(fontsize=9)
plt.tight_layout(); plt.show()
```

Cap-weighted portfolio: return 16.65%, vol 23.28% -> implied risk aversion delta = 2.33
View uncertainty (standard deviation): view 1 7.5%, view 2 5.4%

	cap weight	equilibrium	historical	posterior
AAPL	86.3%	13.55%	17.61%	11.50%
PFE	2.6%	2.07%	-4.92%	3.48%
XOM	11.1%	2.40%	14.16%	4.39%

portfolio	AAPL	PFE	XOM	return	vol	Sharpe
Market cap weights	86.3%	2.6%	11.1%	16.65%	23.28%	0.543
Black-Litterman	66.2%	11.4%	22.4%	14.27%	19.47%	0.527
Tangency on history	52.0%	0.0%	48.0%	15.96%	18.58%	0.644



Interpretation

My equilibrium returns are 13.55%, 2.07% and 2.40%, against historical means of 17.61%, -4.92% and 14.16%. They are never negative and they rank by market weight, so AAPL leads on 86.3% of the cap. Risk aversion of 2.33 falls out of my own data, close to the usual 2.5.

I was surprised my relative view came out bearish. Three points looks like a positive call on AAPL, but equilibrium already had AAPL at 13.55% against XOM at 2.40%, so stating it pulled AAPL down to 11.50% and pushed XOM to 4.39%. The absolute view lifted PFE only to 3.48%, short of the 5% I asserted, because its 5.4% uncertainty makes the posterior split the difference. The weights come out 66.2/11.4/22.4, between the market caps and something balanced, and that costs Sharpe, 0.527 against 0.644.

3 · Estimation error and a robust allocation

My robust portfolio resamples minimum variance, averaging the optimiser over 500 simulated histories from my own moments. The 1pp bump on a single mean separates real hypersensitivity from a plain change of inputs.

```
In [25]: half = len(rets) // 2
train, test = rets.iloc[:half], rets.iloc[half:]

def moments(sample):
    return (expected_returns.mean_historical_return(sample, returns_data=True,
    compounding=False),
            risk_models.sample_cov(sample, returns_data=True))

budget = ({'type': 'eq', 'fun': lambda w: w.sum() - 1.0},)
box = [(0.0, 1.0)] * 3

def tangency_weights(m, S):
    negative_sharpe = lambda w: -(w @ m - RF) / np.sqrt(w @ S @ w)
    return minimize(negative_sharpe, np.ones(3)/3, method='SLSQP', bounds=box,
                    constraints=budget, options={'maxiter': 1000, 'ftol': 1e-14}).x
```

```

def minimum_variance_weights(S):
    return minimize(lambda w: w @ S @ w, np.ones(3)/3, method='SLSQP', bounds=box,
                    constraints=budget, options={'maxiter': 1000, 'ftol': 1e-14}).x

print('Same optimiser, different estimation window:')
print(f"{'':26}{'AAPL':>8}{'PFE':>8}{'XOM':>8}      estimated means")
for label, window in [('full sample',rets), ('first half',train), ('second half',test)]:
    m_w, S_w = moments(window)
    w_t = tangency_weights(m_w.values, S_w.values)
    print(f'  tangency  {label:<14}' + ''.join(f'{x*100:>7.1f}%' for x in w_t) +
          +',', '.join(f'{v*100:>7.1f}%' for v in m_w))
for label, window in [('full sample',rets), ('first half',train), ('second half',test)]:
    m_w, S_w = moments(window)
    print(f'  min-var   {label:<14}' +
          +',', '.join(f'{x*100:>7.1f}%' for x in minimum_variance_weights(S_w.values)))
for label, window in [('full sample',rets), ('first half',train), ('second half',test)]:
    m_w, S_w = moments(window)
    print(f'  risk par   {label:<14}' +
          +',', '.join(f'{x*100:>7.1f}%' for x in equal_risk(S_w.values)))

print('\nSensitivity to a 1pp bump in a single expected return:')
for i, s in enumerate(STOCKS):
    bumped = mu.copy(); bumped.iloc[i] += 0.01
    w_b = optimise(bumped, S_sample, 'max_sharpe')
    print(f'  {s} +1pp      +', '.join(f'{x*100:>7.1f}%' for x in w_b) +
          f'  largest weight shift {np.abs(w_b - w_tan).max()*100:.1f}pp')

DRAWS = 500
rng = np.random.default_rng(42)
acc_minvar, acc_tan, cornered = np.zeros(3), np.zeros(3), 0
for _ in range(DRAWS):
    sim = rng.multivariate_normal(mv/252, Sv/252, size=len(rets))
    m_sim, S_sim = sim.mean(axis=0)*252, np.cov(sim, rowvar=False)*252
    w_sim = tangency_weights(m_sim, S_sim)
    acc_tan += w_sim
    acc_minvar += minimum_variance_weights(S_sim)
    if w_sim.min() < 0.01:
        cornered += 1
w_robust = acc_minvar / DRAWS
w_resamp_ms = acc_tan / DRAWS

print(f'\nResampled over {DRAWS} simulated histories:')
print(f"{'portfolio':<26}{'AAPL':>8}{'PFE':>8}{'XOM':>8}")
for name, w in [('Resampled tangency', w_resamp_ms), ('Resampled min-variance', w_robust),
                ('Shrinkage min-variance', optimise(mu, S_lw, 'min_volatility')),
                ('Sample min-variance', w_minvar), ('Risk parity', w_erc)]:
    print(f'  {name:<24}' + ''.join(f'{x*100:>7.1f}%' for x in w))
print(f'\nDraws where the tangency optimiser pushed a name below 1%: '
      f'{cornered}/{DRAWS} = {cornered/DRAWS:.1f}%')

fig, (axL, axR) = plt.subplots(1, 2, figsize=(13, 4))
x = np.arange(3)
for i, (label, window, colour) in enumerate([('first half', train, BLUE),
                                              ('second half', test, RED)]):
    m_w, S_w = moments(window)
    axL.bar(x + (i-0.5)*0.35, tangency_weights(m_w.values, S_w.values)*100, 0.35,
            color=colour, label=label)
    axR.bar(x + (i-0.5)*0.35, minimum_variance_weights(S_w.values)*100, 0.35,
            color=colour, label=label)
for ax, title in [(axL, 'Tangency weights by estimation window'),
                  (axR, 'Minimum-variance weights by estimation window')]:
    ax.set_xticks(x); ax.set_xticklabels(STOCKS); ax.set_ylabel('Weight (%)')
    ax.set_ylim(0, 100); ax.set_title(title); ax.legend(fontsize=9)
plt.tight_layout(); plt.show()

```

Same optimiser, different estimation window:

		AAPL	PFE	XOM	estimated means
tangency	full sample	52.0%	0.0%	48.0%	+17.6%, -4.9%, +14.2%
tangency	first half	91.2%	0.0%	8.8%	+21.9%, -13.0%, +4.9%
tangency	second half	16.1%	0.0%	83.9%	+13.3%, +3.2%, +23.4%
min-var	full sample	27.4%	33.2%	39.4%	
min-var	first half	36.7%	24.2%	39.1%	
min-var	second half	19.8%	41.4%	38.9%	
risk par	full sample	30.9%	33.3%	35.8%	
risk par	first half	34.9%	29.2%	35.9%	
risk par	second half	28.4%	36.1%	35.5%	

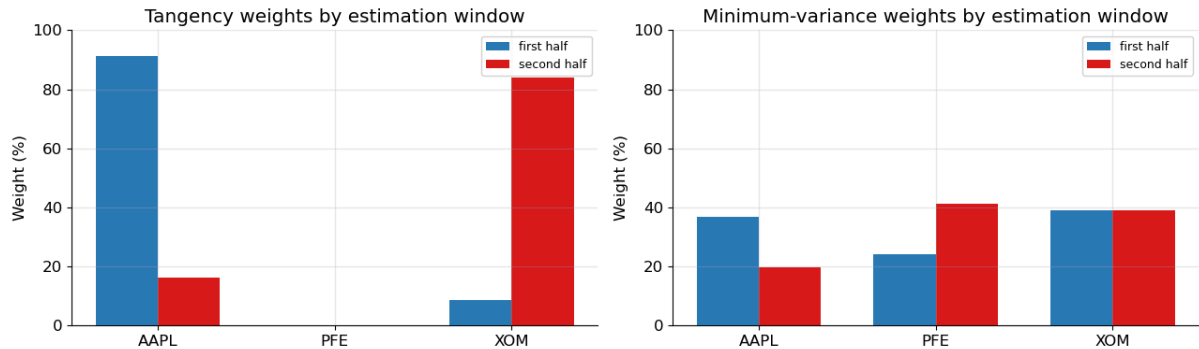
Sensitivity to a 1pp bump in a single expected return:

AAPL +1pp	54.3%	0.0%	45.7%	largest weight shift 2.2pp
PFE +1pp	52.0%	0.0%	48.0%	largest weight shift 0.0pp
XOM +1pp	49.1%	0.0%	50.9%	largest weight shift 2.9pp

Resampled over 500 simulated histories:

portfolio	AAPL	PFE	XOM
Resampled tangency	48.9%	6.2%	44.8%
Resampled min-variance	27.4%	33.3%	39.3%
Shrinkage min-variance	29.4%	33.3%	37.3%
Sample min-variance	27.4%	33.2%	39.4%
Risk parity	30.9%	33.3%	35.8%

Draws where the tangency optimiser pushed a name below 1%: 459/500 = 91.8%



Interpretation

Splitting my sample in half is brutal for tangency. First half it holds 91.2% AAPL and 8.8% XOM, second half 16.1% and 83.9%, the book flipping end to end on the same optimiser. Minimum variance moves less, 36.7% to 19.8% in AAPL, and risk parity least, because neither uses a mean.

The 1pp bump is where my data disagrees with the story that a one-point error moves weights ten or twenty points. Adding 1pp to AAPL moves my largest weight only 2.2pp, to XOM 2.9pp, and to PFE nothing, so the optimiser is not hypersensitive. What breaks it is that the estimate itself is enormous, AAPL's mean going from +21.9% to +13.3% between halves and XOM's from +4.9% to +23.4%. That is also why shrinkage does almost nothing, since with three assets and 750 days my covariance was never the problem. Resampling helps because it targets the mean, averaging tangency to 48.9/6.2/44.8 and putting PFE back, even though 91.8% of single draws still cornered a name. They corner different names, and averaging over that is what I would hold.

4 · ESG tilt

I could not source vendor ratings offline, so these are documented proxy scores I assigned, with the reasoning printed below. Rater A is environment-weighted, rewarding AAPL and marking XOM down for its reserves. Rater B is conduct-weighted, marking PFE down for litigation. Both run 0 to 100, higher better, and I tilt my Black-Litterman allocation since that is what I would otherwise hold.

```
In [26]: esg = pd.read_csv('A3/a3_esg_scores.csv', index_col='ticker').loc[STOCKS]
print(esg[['company', 'rater_A', 'rater_B']].to_string())
for t in STOCKS:
    print(f'\n{t} A {esg.loc[t, "rater_A"]} {esg.loc[t, "basis_A"]}')
    print(f'{t} B {esg.loc[t, "rater_B"]} {esg.loc[t, "basis_B"]}')
rank_corr = spearmanr(esg.rater_A, esg.rater_B).statistic
print(f'\nRank correlation between the two raters: {rank_corr:.2f}')
print(f'Rater A ranks: {" > ".join(esg.rater_A.sort_values(ascending=False).index)}')
print(f'Rater B ranks: {" > ".join(esg.rater_B.sort_values(ascending=False).index)}')

base_sharpe = sharpe(w_bl, mv, Sv)

esg_paths = {}
```

```

for rater in ['rater_A', 'rater_B']:
    scores = esg[rater].values
    start_sc = float(w_bl @ scores)
    best_sc = scores.max()
    print(f'\n{rater}: my allocation scores {start_sc:.1f}, best attainable {best_sc}')
    print(f"  {'floor':>7}{AAPL':>8}{PFE':>8}{XOM':>8}{return':>9}{vol':>8}"
          f"{'Sharpe':>8}{vs base':>9}")
    rows = []
    for step in [0.0, 0.25, 0.50, 0.75, 1.0]:
        floor = start_sc + step * (best_sc - start_sc)
        w_e = optimise(mu_bl, S_bl, 'max_quadratic_utility', risk_aversion=delta,
                      constraint=(lambda w, sc=scores, f=floor: w @ sc >= f))
        sh = sharpe(w_e, mv, Sv)
        rows.append((floor, w_e, sh))
    print(f"  {'floor':>7.1f}" + ''.join(f'x*100:>7.1f}' for x in w_e) +
          f"{'port_return(w_e, mv)*100:>8.2f}'{port_vol(w_e, Sv)*100:>7.2f}"
          f"{'sh:>8.3f}'{sh - base_sharpe:>+9.3f}")
    esg_paths[rater] = rows

fig, axes = plt.subplots(1, 2, figsize=(13, 4), sharey=True)
for ax, rater, colour in [(axes[0], 'rater_A', GREEN), (axes[1], 'rater_B', BLUE)]:
    floors = [r[0] for r in esg_paths[rater]]
    weights = np.array([r[1] for r in esg_paths[rater]]) * 100
    for j, s in enumerate(STOCKS):
        ax.plot(floors, weights[:, j], 'o-', lw=2, label=s,
              color=[BLUE, GOLD, RED][j])
    ax.set_xlabel(f'ESG floor on {rater}'); ax.set_title(f'Weights as the {rater} floor
tightens')
    ax.legend(fontsize=9)
axes[0].set_ylabel('Weight (%)')
plt.tight_layout(); plt.show()

```

	company	rater_A	rater_B
ticker			
AAPL	Apple	82	58
PFE	Pfizer	61	47
XOM	ExxonMobil	24	55

AAPL A 82 Operations run on renewable power with a 2030 supply-chain carbon-neutral pledge; low direct emissions for its size

AAPL B 58 Repeated supply-chain labour and sourcing controversies; concentrated single-region manufacturing dependence

PFE A 61 Moderate direct footprint for a manufacturer; no fossil reserves but energy-intensive production sites

PFE B 47 Large historical litigation and marketing-conduct record including the 2009 off-label settlement; pricing-practice scrutiny

XOM A 24 Fossil-fuel producer carrying reserve-based transition exposure and large financed Scope 3 emissions

XOM B 55 Long-established board and disclosure practices; controversy record concentrated in climate litigation rather than conduct

Rank correlation between the two raters: 0.50

Rater A ranks: AAPL > PFE > XOM

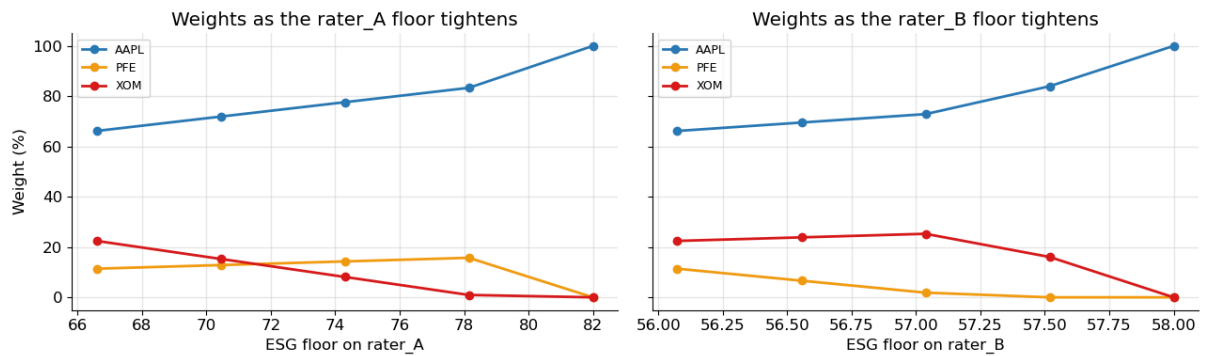
Rater B ranks: AAPL > XOM > PFE

rater_A: my allocation scores 66.6, best attainable 82

floor	AAPL	PFE	XOM	return	vol	Sharpe	vs base
66.6	66.2%	11.4%	22.4%	14.27%	19.47%	0.527	+0.000
70.4	71.9%	12.8%	15.3%	14.19%	20.48%	0.498	-0.030
74.3	77.6%	14.3%	8.1%	14.11%	21.65%	0.467	-0.060
78.1	83.3%	15.7%	0.9%	14.03%	22.96%	0.437	-0.090
82.0	100.0%	0.0%	0.0%	17.61%	26.32%	0.517	-0.010

rater_B: my allocation scores 56.1, best attainable 58

floor	AAPL	PFE	XOM	return	vol	Sharpe	vs base
56.1	66.2%	11.4%	22.4%	14.27%	19.47%	0.527	+0.000
56.6	69.5%	6.6%	23.9%	15.30%	20.08%	0.562	+0.035
57.0	72.9%	1.9%	25.3%	16.32%	20.76%	0.593	+0.066
57.5	84.0%	0.0%	16.0%	17.06%	22.81%	0.572	+0.045
58.0	100.0%	0.0%	0.0%	17.61%	26.32%	0.517	-0.010



Interpretation

My two raters agree on AAPL and disagree on everything else, with a rank correlation of 0.50. Rater A orders them AAPL > PFE > XOM and rater B AAPL > XOM > PFE, so the bottom two swap, as real providers do.

The consequence is sharper than I expected. Tightening the rater A floor from 66.6 to 78.1 cuts XOM from 22.4% to 0.9%, while tightening rater B from 56.1 to 57.0 cuts PFE to 1.9% as XOM rises to 25.3%. Same constraint, and the two raters tell me to sell opposite names. The cost flips too, from 0.527 down to 0.437 on rater A but up to 0.593 on rater B. So whether an ESG screen is expensive is not about ESG, it is about which provider I signed up with, and a 0.50 correlation makes that close to a coin flip. At the maximum floor both collapse to 100% AAPL, so a screen tight enough to bind on three names is a concentration decision, not a tilt.

5 · Scenario stress testing

Each scenario is a market move through beta plus a firm-specific overlay. Severely adverse is a broad equity shock, the climate transition puts a carbon price and stranded reserves on XOM, and the policy shock pairs drug-pricing reform with tariffs. The historical window is the worst 20-day stretch my book ever had.

```
In [27]: betas = {}
for s in STOCKS:
    aligned = sm.add_constant(mkt).join(rets[s]).dropna()
    fit = sm.OLS(aligned[s], aligned[['const', BENCH]]).fit()
    betas[s] = fit.params[BENCH]
    print(f'{s}  beta {fit.params[BENCH]:.3f}  (t = {fit.tvalues[BENCH]:.1f}, '
          f'R-squared {fit.rsquared:.3f})')
betas = pd.Series(betas)

SCENARIOS = {
    'Severely adverse': (-0.25, {'AAPL': 0.00, 'PFE': 0.00, 'XOM': 0.00}),
    'Climate transition': (-0.15, {'AAPL': -0.02, 'PFE': -0.02, 'XOM': -0.25}),
    'Drug pricing + tariffs': (-0.08, {'AAPL': -0.12, 'PFE': -0.20, 'XOM': 0.03}),
}

shock_table = pd.DataFrame({name: {s: betas[s]*market + overlay[s] for s in STOCKS}
                           for name, (market, overlay) in SCENARIOS.items()}).T[STOCKS]

rolling20 = port_eq.rolling(20).sum()
window_end = rolling20.idxmin()
window_start = port_eq.index[port_eq.index.get_loc(window_end) - 19]
window_rets = rets.loc[window_start:window_end]
hist_shock = (1 + window_rets).prod() - 1

print(f'\nPer-stock shocks (beta times the market move, plus the overlay):')
print(shock_table.mul(100).round(1).to_string())
print(f'\nWorst 20-day window for the equal-weight book: '
      f'{window_start.date()} to {window_end.date()}')
print(f' cumulative {(((1 + (window_rets * W_EQ).sum(axis=1)).prod() - 1)*100:.2f)}%, '
```

```

    f'worst single day {(window_rets * W_EQ).sum(axis=1).min()*100:.2f}%')
print(' per stock: ' + ', '.join(f'{s} {hist_shock[s]*100:+.1f}%' for s in STOCKS))

window_port = (window_rets * W_EQ).sum(axis=1)
stress_var = -np.percentile(window_port, 1)
stress_es = -window_port[window_port <= np.percentile(window_port, 2.5)].mean()
print(f'\n{"":24}{"1-day VaR99":>13}{"1-day ES97.5":>14}{"ann vol":>10}')
print(f'{"Normal times":24}{var99*100:>12.2f}%{es975*100:>13.2f}%'
      f'{port_eq.std()*np.sqrt(252)*100:>9.2f}%')
print(f'{"Stress window":24}{stress_var*100:>12.2f}%{stress_es*100:>13.2f}%'
      f'{window_port.std()*np.sqrt(252)*100:>9.2f}%')
print(f'{"Ratio":24}{stress_var/var99:>12.2f}x{stress_es/es975:>13.2f}x'
      f'{window_port.std()/port_eq.std():>9.2f}x')

vol_multiple = window_port.std() / port_eq.std()
horizon = len(window_port)
centred = port_eq - port_eq.mean()

print(f'\nEqual-weight book under each scenario, risk re-measured at '
      f'{vol_multiple:.2f}x normal volatility spread over {horizon} days:')
print(f' {"":24}{"P&L":>9}{"vs VaR99":>10}{"VaR99":>9}{"ES97.5":>9}{"vs normal":>11}')
for name in list(SCENARIOS) + ['Historical replay']:
    shock = hist_shock if name == 'Historical replay' else shock_table.loc[name]
    pnl = float(shock @ W_EQ)
    stressed = pnl / horizon + centred * vol_multiple
    scenario_var = -np.percentile(stressed, 1)
    scenario_es = -stressed[stressed <= np.percentile(stressed, 2.5)].mean()
    print(f' {name:<24}{pnl*100:>8.2f}%{abs(pnl)/var99:>9.1f}x'
          f'{scenario_var*100:>8.2f}%{scenario_es*100:>8.2f}%{scenario_var/var99:>10.2f}x')
print(f' {"Normal times":<24}{"":>9}{"":>10}{var99*100:>8.2f}%{es975*100:>8.2f}%'
      f'{1.0:>10.2f}x')

fig, (axL, axR) = plt.subplots(1, 2, figsize=(13, 4))
x = np.arange(3)
for i, name in enumerate(SCENARIOS):
    axL.bar(x + (i-1)*0.27, shock_table.loc[name]*100, 0.27,
           color=[RED, GREEN, GOLD][i], label=name)
axL.axhline(0, color='black', lw=0.8)
axL.set_xticks(x); axL.set_xticklabels(STOCKS); axL.set_ylabel('Shock (%)')
axL.set_title('Per-stock shocks by scenario'); axL.legend(fontsize=8)
cum = (1 + window_port).cumprod() - 1
axR.fill_between(cum.index, cum*100, color=RED, alpha=0.3)
axR.plot(cum.index, cum*100, color=RED, lw=2)
axR.axhline(0, color='black', lw=0.8)
axR.set_ylabel('Cumulative return (%)'); axR.tick_params(axis='x', rotation=20)
axR.set_title(f'Worst 20 days in my own sample')
plt.tight_layout(); plt.show()

```

AAPL beta 1.127 (t = 23.0, R-squared 0.414)
 PFE beta 0.386 (t = 6.8, R-squared 0.058)
 XOM beta 0.238 (t = 4.3, R-squared 0.024)

Per-stock shocks (beta times the market move, plus the overlay):

	AAPL	PFE	XOM
Severely adverse	-28.2	-9.7	-6.0
Climate transition	-18.9	-7.8	-28.6
Drug pricing + tariffs	-21.0	-23.1	1.1

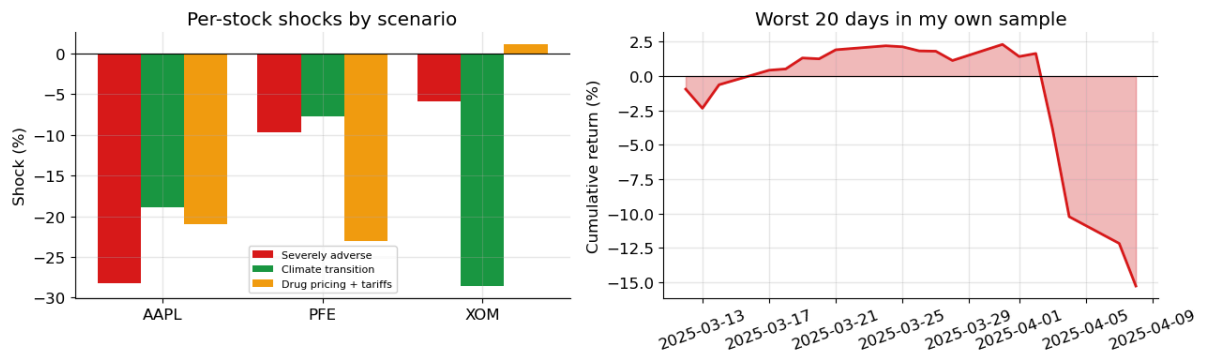
Worst 20-day window for the equal-weight book: 2025-03-12 to 2025-04-08

cumulative -15.26%, worst single day -6.64%
 per stock: AAPL -21.9%, PFE -16.0%, XOM -7.7%

	1-day VaR99	1-day ES97.5	ann vol
Normal times	2.42%	2.89%	16.08%
Stress window	6.40%	6.64%	34.22%
Ratio	2.64x	2.30x	2.13x

Equal-weight book under each scenario, risk re-measured at 2.13x normal volatility spread over 20 days:

	P&L	vs VaR99	VaR99	ES97.5	vs normal
Severely adverse	-14.59%	6.0x	5.96%	6.96%	2.46x
Climate transition	-18.42%	7.6x	6.15%	7.15%	2.54x
Drug pricing + tariffs	-14.34%	5.9x	5.95%	6.94%	2.45x
Historical replay	-15.19%	6.3x	5.99%	6.99%	2.47x
Normal times			2.42%	2.89%	1.00x



Interpretation

My betas do most of the work and they are very unequal. AAPL is 1.127 (R-squared 0.414), XOM only 0.238 (0.024), so 98% of what moves XOM is not the market. A 25% market fall takes XOM down just 6.0%, so anything that threatens it has to come through the overlay, which is why I design scenarios, not factor shocks.

Climate transition is my worst case at -18.42%, beating the severely adverse -14.59% on a smaller market shock, because a concentrated hit on the name with least market exposure does more than a broad one. My worst actual 20 days ran 12 March to 8 April 2025 and cost 15.26%, so my scenarios are severe without being fantasy. Inside that window my 1-day 99% VaR is 6.40% against 2.42% normally, so a scenario costs twice, moving the book 5.9 to 7.6 times my daily VaR and leaving the VaR itself near 6%.

These do not capture system-wide stress. Each is one market move plus independent firm overlays, so the only thing linking my stocks is beta, and XOM's is 0.238. That cannot produce the 0.689 crisis correlation I measure in my worst 20 days, the correlated failure macroprudential regulation exists for.

6 · Systemic risk, contagion and model risk

Delta-CoVaR asks how much deeper the market's 5% tail gets when a name is at its own 5% tail. I also split the sample by how bad the market day was, because my whole frontier rests on low correlations holding up.

```
In [28]: ALPHA = 0.05
covar_rows = {}
for s in STOCKS:
    common = rets[s].dropna().index.intersection(mkt.dropna().index)
    y, X = mkt.loc[common].values, rets[s].loc[common].values
    fit = QuantReg(y, np.column_stack([np.ones(len(X)), X])).fit(q=ALPHA, max_iter=5000)
    intercept, slope = fit.params
    covar_distress = intercept + slope * np.percentile(X, ALPHA*100)
    covar_normal = intercept + slope * np.median(X)
    covar_rows[s] = {'stock VaR5 %': np.percentile(X, ALPHA*100)*100,
                    'quantile beta': slope,
                    'CoVaR distress %': covar_distress*100,
                    'CoVaR normal %': covar_normal*100,
                    'Delta-CoVaR %': (covar_distress - covar_normal)*100}
covar_df = pd.DataFrame(covar_rows).T
print('CoVaR of the index conditional on each stock (quantile regression at 5%):')
print(covar_df.round(3).to_string())

market_tail = np.percentile(mkt, 5)
tail_days = mkt[mkt <= market_tail]
print(f'\nMarginal expected shortfall on the market\'s worst 5% of days '
      f'({len(tail_days)} days, threshold {market_tail*100:.2f}%):')
for s in STOCKS:
```

```

print(f' {s} {rets[s].loc[tail_days.index].mean()*100:.3f}%')

decile = np.percentile(mkt, 10)
crisis = rets.loc[mkt[mkt <= decile].index]
calm = rets.loc[mkt[mkt > decile].index]
def average_pairwise(c):
    v = c.values
    return (v[0, 1] + v[0, 2] + v[1, 2]) / 3
print(f'\nCorrelation on the market\'s worst decile ({len(crisis)} days) vs the rest:')
print(f' calm average pairwise {average_pairwise(calm.corr()):.3f}')
print(calm.corr().round(3).to_string())
print(f' crisis average pairwise {average_pairwise(crisis.corr()):.3f}')
print(crisis.corr().round(3).to_string())
print(f'\nAlso in the worst 20-day window: average pairwise '
      f'{average_pairwise(window_rets.corr()):.3f}')
print(f'Equal-weight annualised volatility: calm '
      f'{(calm * W_EQ).sum(axis=1).std()*np.sqrt(252)*100:.1f}%, crisis '
      f'{(crisis * W_EQ).sum(axis=1).std()*np.sqrt(252)*100:.1f}%')

fig, (axL, axR) = plt.subplots(1, 2, figsize=(13, 4))
axL.barh(STOCKS, covar_df['Delta-CoVaR %'].values, color=RED)
axL.set_xlabel('Delta-CoVaR (%)'); axL.set_title('Systemic contribution (more negative = larger)')
labels = ['AAPL-PFE', 'AAPL-XOM', 'PFE-XOM']
calm_c = [calm.corr().values[0,1], calm.corr().values[0,2], calm.corr().values[1,2]]
crisis_c = [crisis.corr().values[0,1], crisis.corr().values[0,2], crisis.corr().values[1,2]]
x = np.arange(3)
axR.bar(x - 0.2, calm_c, 0.4, color=BLUE, label='Calm days')
axR.bar(x + 0.2, crisis_c, 0.4, color=RED, label='Worst decile')
axR.set_xticks(x); axR.set_xticklabels(labels); axR.set_ylabel('Correlation')
axR.set_title('Diversification on the days it matters'); axR.legend(fontsize=9)
plt.tight_layout(); plt.show()

```

CoVaR of the index conditional on each stock (quantile regression at 5%):

	stock VaR5 %	quantile beta	CoVaR distress %	CoVaR normal %	Delta-CoVaR %
AAPL	-2.534	0.393	-2.136	-1.095	-1.041
PFE	-2.339	0.168	-1.747	-1.357	-0.390
XOM	-2.381	0.152	-1.789	-1.414	-0.375

Marginal expected shortfall on the market's worst 5% of days (38 days, threshold -1.46%):

AAPL	-2.358%
PFE	-1.002%
XOM	-0.818%

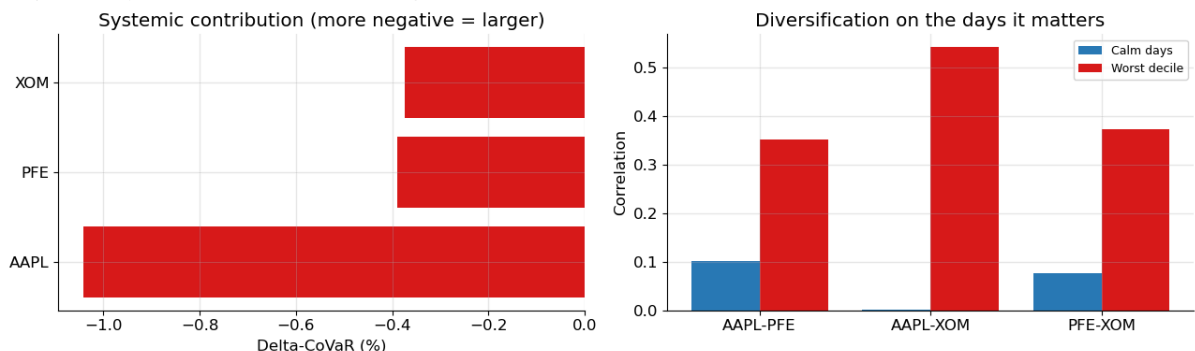
Correlation on the market's worst decile (75 days) vs the rest:

	calm	average pairwise	0.060
	AAPL	PFE	XOM
AAPL	1.000	0.102	0.001
PFE	0.102	1.000	0.076
XOM	0.001	0.076	1.000

	crisis	average pairwise	0.423
	AAPL	PFE	XOM
AAPL	1.000	0.353	0.542
PFE	0.353	1.000	0.373
XOM	0.542	0.373	1.000

Also in the worst 20-day window: average pairwise 0.689

Equal-weight annualised volatility: calm 14.3%, crisis 22.0%



Interpretation

AAPL is my systemic name and it is not close. Its Delta-CoVaR is -1.041% against -0.390% for PFE and -0.375% for XOM, so when AAPL is at its own 5% tail the index tail deepens nearly three times as much. Yet all three have almost identical standalone

risk, own 5% VaRs of -2.53%, -2.34% and -2.38%, so volatility alone would rank them equal. Marginal expected shortfall agrees, AAPL losing 2.358% on the market's worst days against 0.818% for XOM.

Then the correlations undo much of what came before. On calm days my average pairwise correlation is 0.060, but on the worst market decile it is 0.423, and in my worst 20-day window it reaches 0.689, book volatility rising from 14.3% to 22.0%. All three fall together on exactly the days I need them not to, so the low correlations my frontier leaned on are only a calm-market property. That is also where my models fail. The quantile regression is linear, assuming the same tail slope in a crash as on an ordinary day, on a 5% quantile resting on only 38 of my 750 days.

7 · Resilience, the out-of-sample test and what I would actually hold

Every portfolio is rebuilt from scratch on the training half, prior and ESG floor included, so nothing from the test period leaks in.

```
In [29]: def build_portfolios(sample):
m, S_s = moments(sample)
S_shrunk = risk_models.CovarianceShrinkage(sample, returns_data=True).ledoit_wolf()
r_m = (sample * w_mkt.values).sum(axis=1)
d = (r_m.mean()*252 - RF) / (r_m.std()*2 * 252)
m_bl, S_b = black_litterman(w_mkt, S_shrunk, P, Q,
                             proportional_prior(S_shrunk, TAU, P), d, TAU)
utility = lambda constraint=None: optimise(m_bl, S_b, 'max_quadratic_utility',
                                             risk_aversion=d, constraint=constraint)

gen = np.random.default_rng(42)
accumulated = np.zeros(3)
for _ in range(500):
    sim = pd.DataFrame(gen.multivariate_normal(m.values/252, S_s.values/252,
                                                size=len(sample)), columns=STOCKS)
    accumulated += optimise(*moments(sim), 'min_volatility')
floor_a = float(utility() @ esg.rater_A.values)
floor_a += 0.50 * (esg.rater_A.max() - floor_a)
return {'Equal weight': W_EQ,
        'Minimum variance': optimise(m, S_s, 'min_volatility'),
        'Tangency': optimise(m, S_s, 'max_sharpe'),
        'Black-Litterman': utility(),
        'Risk parity': equal_risk(S_s.values),
        'Robust (resampled)': accumulated / 500,
        'ESG-constrained': utility(lambda w: w @ esg.rater_A.values >= floor_a)}

candidates = build_portfolios(rets)
columns = ['Severely adverse', 'Climate', 'Policy shock', 'Historical']
print(f'{"portfolio":<22}{AAPL':>8}{PFE':>8}{XOM':>8}" +
      "'.join(f'{"c":>18}' for c in columns))
for name, w in candidates.items():
    losses = [float(shock_table.loc[k] @ w) for k in SCENARIOS] + [float(hist_shock @ w)]
    print(f'{"name":<22}' + "'.join(f'{"x*100":>7.1f}%' for x in w) +
          "'.join(f'{"v*100":>17.2f}%' for v in losses))
worst = {name: min([float(shock_table.loc[k] @ w) for k in SCENARIOS] + [float(hist_shock @ w)])
          for name, w in candidates.items()}
print(f'\nMost resilient on its own worst case: '
      f'{"max(worst, key=worst.get)}' at {"max(worst.values())*100:.2f}%')
print(f'Least resilient: {"min(worst, key=worst.get)}' at {"min(worst.values())*100:.2f}%')

crisis_cov = crisis.cov() * 252
mu_bl_stress, S_bl_stress = black_litterman(w_mkt, crisis_cov, P, Q,
                                             proportional_prior(crisis_cov, TAU, P), delta, TAU)
stressed_reopt = {'Minimum variance': (w_minvar, optimise(mu, crisis_cov, 'min_volatility')),
                  'Tangency': (w_tan, optimise(mu, crisis_cov, 'max_sharpe')),
                  'Risk parity': (w_erc, equal_risk(crisis_cov.values)),
                  'Black-Litterman': (w_bl, optimise(mu_bl_stress, S_bl_stress,
                                                       'max_quadratic_utility',
                                                       risk_aversion=delta))}
print('\nRe-optimising on my crisis-window covariance (base means held):')
print(f'{"portfolio":<20}{base wt':>20}{stressed wt':>22}{move':>9}")
```

```

for name, (wb, ws) in stressed_reopt.items():
    move = np.abs(np.asarray(ws) - np.asarray(wb)).sum()
    print(f'{name:<20}' + '/' + f'{x*100:.0f}' for x in wb).rjust(19) +
        ' ' + '/' + f'{x*100:.0f}' for x in ws).rjust(20) + f'{move*100:>7.1f}pp')

trained = build_portfolios(train)
tested = build_portfolios(test)
print(f'\nOut-of-sample: weights fitted on {train.index[0].date()} to '
      f'{train.index[-1].date()}, scored on {test.index[0].date()} to {test.index[-1].date()}')
print(f'{"portfolio":<22}{return':>9}{vol':>8}{Sharpe':>8}{max DD':>9}{vs 1/N':>9}
      {'turnover':>10}")
oos = {}
for name, w in trained.items():
    r = (test * w).sum(axis=1)
    ann_r, ann_v = r.mean()*252, r.std()*np.sqrt(252)
    cumulative = (1 + r).cumprod()
    drawdown = ((cumulative - cumulative.cummax()) / cumulative.cummax()).min()
    turnover = np.abs(np.asarray(w) - np.asarray(tested[name])).sum()
    oos[name] = {'ret': ann_r, 'vol': ann_v, 'sharpe': (ann_r - RF)/ann_v,
                'dd': drawdown, 'to': turnover}
for name, m_ in oos.items():
    print(f'{name:<22}{m_['ret']*100:>8.2f}%{m_['vol']*100:>7.2f}%{m_['sharpe']:>8.3f}'
          f'{m_['dd']*100:>8.2f}%{m_['sharpe']-oos['Equal weight']['sharpe']:>9.3f}
          {m_['to']:>9.2f}')
beat = [n for n, m_ in oos.items() if m_['sharpe'] > oos['Equal weight']['sharpe']]
print(f'\nBeat 1/N out of sample: {"", ".join(beat) if beat else "none"}')

fig, (axL, axR) = plt.subplots(1, 2, figsize=(13, 4))
names = list(candidates)
axL.barh(names, [worst[n]*100 for n in names],
         color=[GREEN if worst[n] == max(worst.values()) else
               RED if worst[n] == min(worst.values()) else BLUE for n in names])
axL.set_xlabel('Worst-case loss across all scenarios (%)')
axL.set_title('Scenario resilience')
for name in names:
    axR.plot((1 + (test * trained[name]).sum(axis=1)).cumprod(), lw=1.8,
            color=GREY if name == 'Equal weight' else None,
            ls='--' if name == 'Equal weight' else '-', label=name)
axR.set_title('Out-of-sample growth of 1'); axR.legend(fontsize=7); axR.tick_params(axis='x',
rotation=20)
plt.tight_layout(); plt.show()

```

portfolio	AAPL	PFE	XOM	Severely adverse	Climate	Policy sho
ck Historical						
Equal weight	33.3%	33.3%	33.3%	-14.59%	-18.42%	-14.3
4% -15.19%						
Minimum variance	27.4%	33.2%	39.4%	-13.27%	-19.02%	-13.0
0% -14.34%						
Tangency	52.0%	0.0%	48.0%	-17.52%	-23.54%	-10.4
1% -15.10%						
Black-Litterman	66.2%	11.4%	22.4%	-21.08%	-19.81%	-16.2
9% -18.05%						
Risk parity	30.9%	33.3%	35.8%	-14.05%	-18.67%	-13.7
8% -14.84%						
Robust (resampled)	27.4%	33.3%	39.3%	-13.27%	-19.01%	-13.0
1% -14.34%						
ESG-constrained	77.6%	14.3%	8.1%	-23.73%	-18.10%	-19.5
2% -19.92%						

Most resilient on its own worst case: Equal weight at -18.42%

Least resilient: ESG-constrained at -23.73%

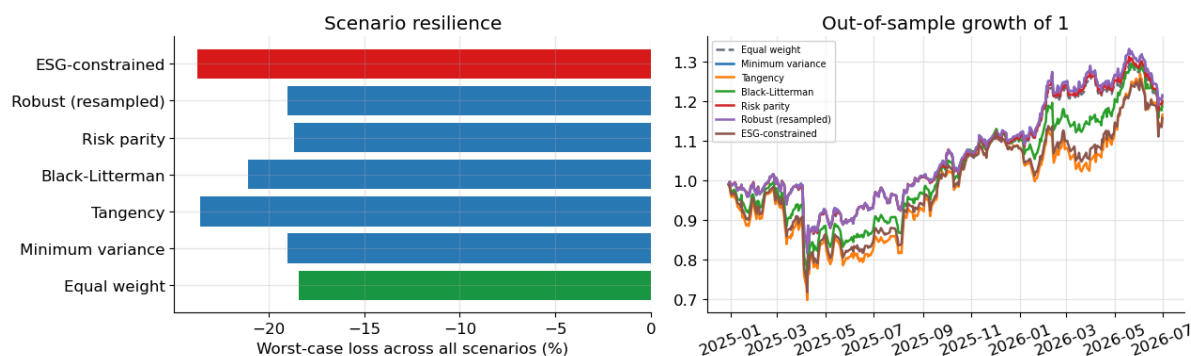
Re-optimising on my crisis-window covariance (base means held):

portfolio	base wt	stressed wt	move
Minimum variance	27/33/39	26/52/22	37.6pp
Tangency	52/0/48	76/0/24	48.4pp
Risk parity	31/33/36	31/38/30	11.0pp
Black-Litterman	66/11/22	76/3/21	19.0pp

Out-of-sample: weights fitted on 2023-07-05 to 2024-12-27, scored on 2024-12-30 to 2026-06-30

portfolio	return	vol	Sharpe	max DD	vs 1/N	turnover
Equal weight	13.31%	18.46%	0.504	-19.51%	+0.000	0.00
Minimum variance	14.81%	18.88%	0.573	-19.49%	+0.069	0.34
Tangency	14.23%	28.28%	0.362	-29.40%	-0.143	1.50
Black-Litterman	14.76%	23.08%	0.466	-24.49%	-0.038	0.12
Risk parity	13.98%	18.61%	0.537	-19.51%	+0.032	0.14
Robust (resampled)	14.84%	18.89%	0.574	-19.50%	+0.069	0.35
ESG-constrained	13.02%	25.47%	0.354	-27.39%	-0.150	0.12

Beat 1/N out of sample: Minimum variance, Risk parity, Robust (resampled)



Interpretation

My two tests disagree, and that is the finding. On worst-case loss equal weight wins at -18.42%, ahead of risk parity at -18.67%, while Black-Litterman loses 21.08% and ESG 23.73%. Out of sample the order flips. Minimum variance and the resampled book reach 0.573 and 0.574 against 1/N's 0.504, risk parity 0.537, while tangency, Black-Litterman and ESG all lose, tangency at 0.362. So every construction that ignores expected returns beats 1/N and every one that uses them loses, the DeMiguel result on three stocks. Turnover says why. Re-estimating on the second half moves tangency 1.50 in absolute weight but risk parity only 0.14, and 1/N nothing, so the winners barely trade and the one estimation error sinks trades most.

Stress says the same. Minimum variance even loses to equal weight in my climate scenario, -19.02% against -18.42%, holding 39.4% XOM for low volatility when XOM carries my transition exposure. Re-optimising on my crisis-window covariance, minimum variance reshuffles 37.6pp and tangency 48.4pp while risk parity moves only 11.0pp, so the two that scored best are the least stable once the correlations I criticised change.

So I would hold risk parity, its 0.537 Sharpe near the best, barely trading and barely moving under stress, and needing no expected-return estimate. I would not hold my ESG version, which loses to 1/N. Against Basel III the shape is right and the depth is not, since I have the scenarios and losses but no capital stack and only one horizon, and every number came from one person running one set of code, so governance needs independent validation and a model inventory.